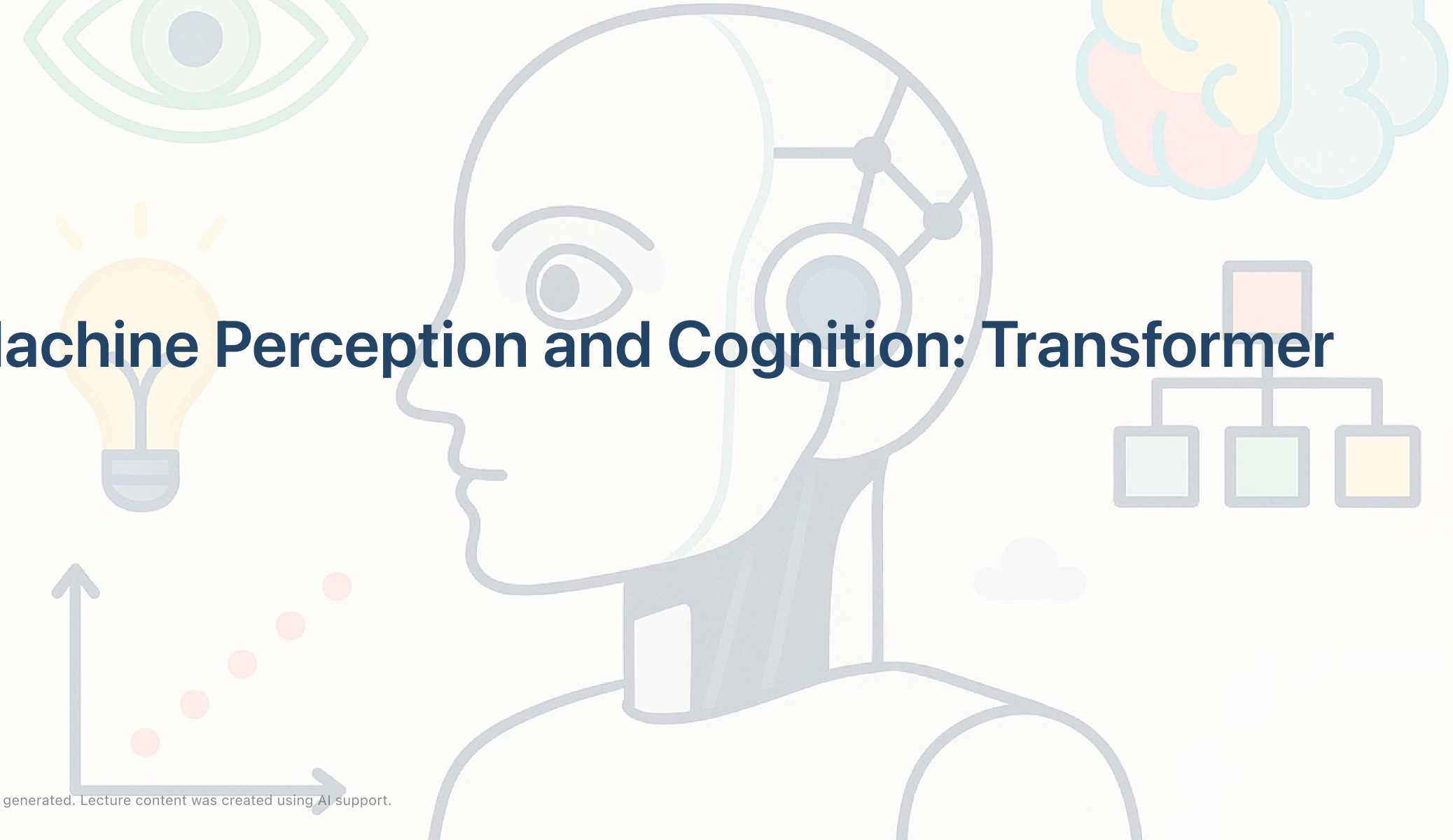
Technische Hochschule Ingolstadt

Machine Perception and Cognition: Transformer



Cover image is AI generated. Lecture content was created using AI support.

From RNNs to Transformers

Recurrent neural networks process sequences **step by step** — each hidden state depends on the previous one:

- **Sequential bottleneck**: processing of step t must wait for step $t-1$ — time steps must be computed sequentially during training
- **Vanishing gradients**: long-range dependencies are hard to capture even with LSTM
- **Fixed context**: the hidden state has limited capacity to carry information over very long sequences

From RNNs to Transformers

Transformer (Vaswani et al., 2017) addresses all three limitations:

- Processes the **entire sequence in parallel** using self-attention
- Connects **any two positions** directly — no intermediate steps
- Scales effectively to large models through parallel processing of sequence positions and efficient GPU utilization

Key Ideas of the Transformer

The Transformer is built on three fundamental ideas:

1. **Token embeddings** — represent discrete symbols as dense continuous vectors
2. **Positional encoding** — inject order information, since attention without positions is permutation-equivariant
3. **Multi-head self-attention** — each position directly attends to all others in parallel

These ideas are combined in stacked **encoder** and **decoder** blocks, each with residual connections and layer normalization, forming a flexible and highly scalable architecture.

What is Tokenization?

Before a neural network can process text, raw characters must be converted to discrete **tokens**:

$$\text{raw text} \xrightarrow{\text{tokenizer}} \text{sequence of token IDs} \xrightarrow{\text{embedding}} \text{sequence of vectors}$$

A **token** is the basic unit of text the model sees. It can be:

- A full word: `"transformer"` → 1 token
- A subword unit: `"transformer"` → `["transform", "er"]` → 2 tokens
- A single character

Most modern large language models use **subword tokenization**, which balances vocabulary size with coverage of rare and unseen words.

Word-Level vs. Subword Tokenization

Property	Word-level	Subword (BPE)
Vocabulary size	Very large (millions)	Controlled (30k–100k)
Unknown words	Frequent	Rare
Morphology	Ignored	Partially captured
Efficiency	Low for rare words	High

Problem with word-level: `"unhappy"`, `"unhappiness"`, `"unhappily"` are three unrelated tokens — the shared stem `"happy"` and prefix `"un"` are lost.

Subword tokenization decomposes words into reusable pieces: `["un", "happy", "ness"]` — shared morphemes are represented by shared tokens.

Byte Pair Encoding (BPE)

Byte Pair Encoding (Sennrich et al., 2016) is a widely used subword tokenization algorithm:

Algorithm:

1. Start with a character-level vocabulary
2. Count all adjacent symbol pairs in the training corpus
3. Merge the most frequent pair into a new symbol
4. Repeat until the vocabulary reaches the target size

Typical subword vocabularies used in practice range from **30,000 to 100,000 tokens**, balancing coverage with model size.

The Vocabulary

The tokenizer produces a **vocabulary** $\mathcal{V}$ — a finite set of tokens with size $|\mathcal{V}|$.

Each token is assigned an integer ID during tokenizer construction, for example:

$$\text{"the"} \mapsto 464, \quad \text{"cat"} \mapsto 3797, \quad \text{"sat"} \mapsto 3332$$

Special tokens are also included: `[BOS]` (beginning of sequence), `[EOS]` (end of sequence), `[PAD]` (padding), `[UNK]` (unknown).

A sentence is encoded as a sequence of IDs, forming the input to the embedding layer:

$$\text{"The cat sat"} \mapsto [464,\ 3797,\ 3332]$$

Token Embeddings

Token IDs are discrete integers. A **token embedding** maps each ID to a dense real-valued vector:

$$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$$

The embedding matrix $\mathbf{E}$ has one row per vocabulary token. For a token with ID k, the embedding is the k-th row:

$$\mathbf{x}_k = \mathbf{E}[k, :] \in \mathbb{R}^d$$

This is a **linear** lookup — during training, the rows of $\mathbf{E}$ are updated by gradient descent to learn useful vector representations that capture semantic and syntactic relationships between tokens.

The Embedding Matrix

The embedding dimension d is a key hyperparameter — typical values range from $d = 512$ for small research models up to many thousands for very large models.

For an input sequence of n tokens, the embedding layer produces a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{pmatrix}$$

The Ordering Problem

Attention without positional information is **permutation-equivariant**: if the input tokens are reordered, the outputs are reordered in the same way.

For a permutation matrix $\mathbf{P}$:

$$\text{Attention}(\mathbf{PX}) = \mathbf{P}\,\text{Attention}(\mathbf{X})$$

Without additional information, the Transformer cannot tell whether token i appears before or after token j.

Solution: add a **positional encoding** vector to each token embedding before feeding the sequence into the attention layers. This injects order information into the representations.

Sinusoidal Positional Encoding

The original Transformer (Vaswani et al., 2017) uses **deterministic sinusoidal encoding**. For position $pos \in \{0, 1, \ldots, n-1\}$ and embedding dimension index $i \in \{0, 1, \ldots, d/2 - 1\}$:

$$PE(pos,\, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos,\, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- The denominator $10000^{2i/d}$ spans a wide range of frequencies across the d dimensions
- **No parameters are learned** — the encoding is fixed and added element-wise to the token embeddings: $\mathbf{z}_i = \mathbf{x}_i + PE(i, :)$

Sinusoidal Positional Encoding

The same sinusoidal formula is used in diffusion models to encode the **noise timestep** t (as covered in the diffusion models chapter):

$$\text{emb}(t)_{2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{emb}(t)_{2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

In both cases, the 10000 is a fixed scale hyperparameter that spreads sinusoidal frequencies across the embedding dimensions. As the dimension index i increases, the denominator $10000^{2i/d}$ increases, resulting in lower-frequency components. Thus, lower dimensions encode fast-changing patterns while higher dimensions encode slower-varying patterns.

Positional Encoding

Absolute positional sinusoidal encoding:

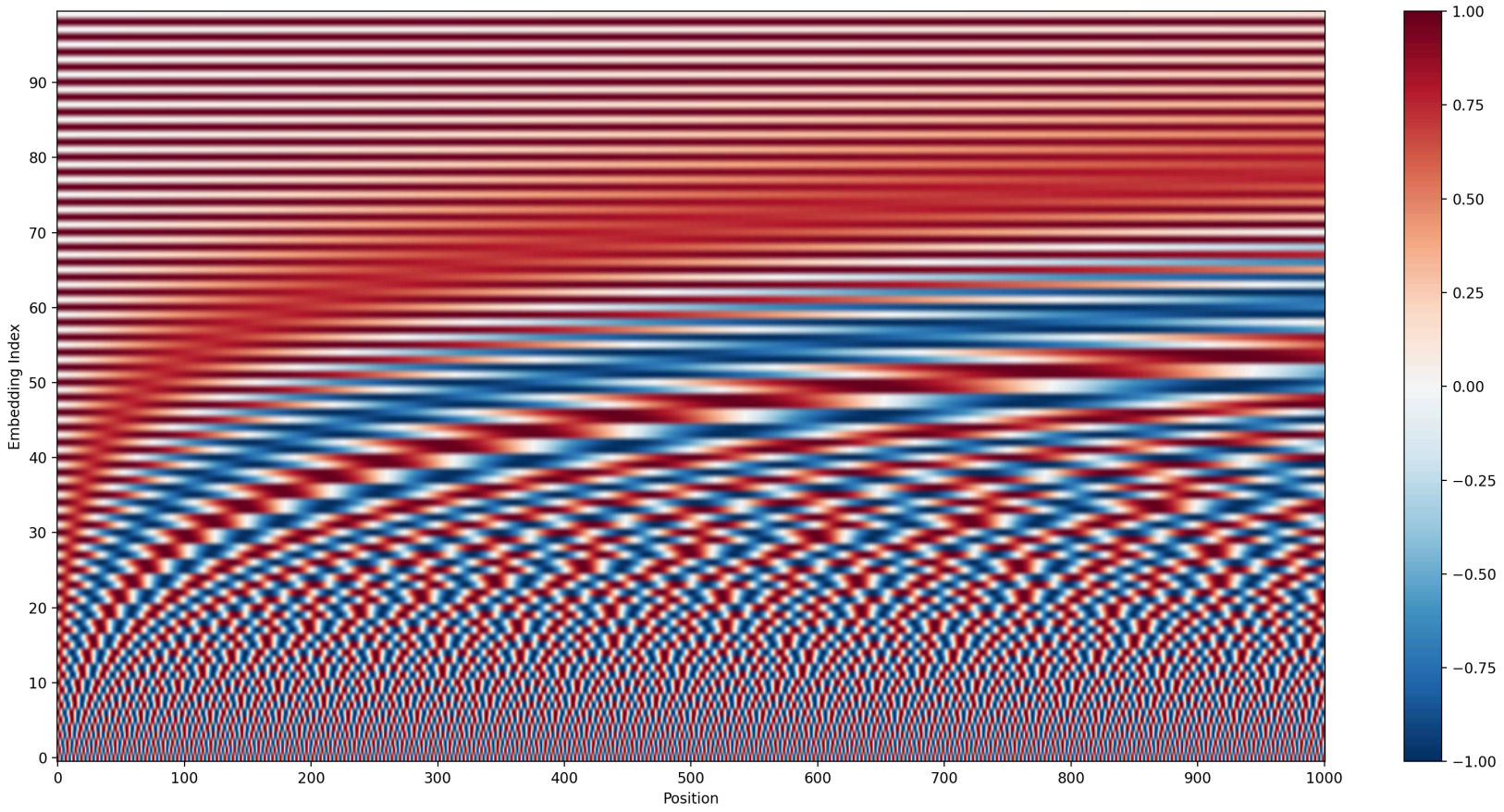


Image source: Nils Blümer, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Absolute_positional_encoding.png

Sinusoidal Encoding Properties

Key properties of sinusoidal positional encoding:

- **Unique over practical sequence lengths**: each position pos receives a distinct encoding vector in $\mathbb{R}^d$
- **Bounded**: all values are in $[-1, 1]$ — no scaling issues regardless of sequence length
- **Can be evaluated beyond training length**: the formula applies to any pos, even positions not seen during training, although performance at significantly longer sequence lengths may degrade due to distribution shift

Transformer Networks

A standard transformer with embedding, positional encoding, encoder-decoder blocks and linear prediction:

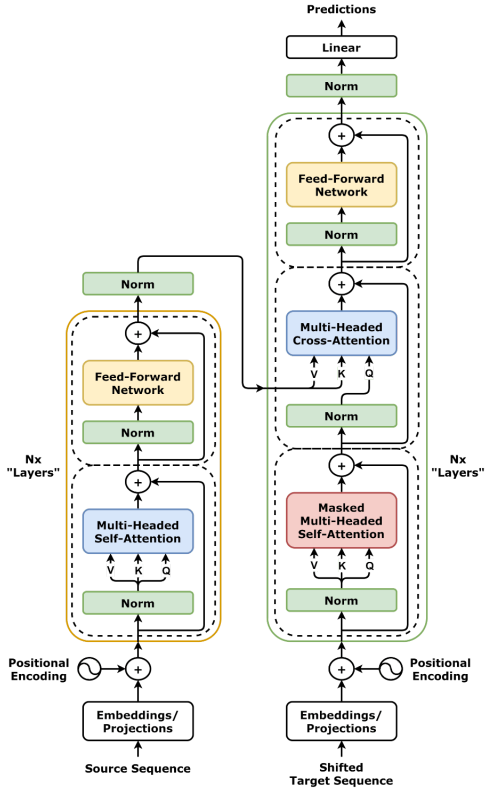


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_full_architecture.png

One Encoder Layer

An encoder layer with self-attention:

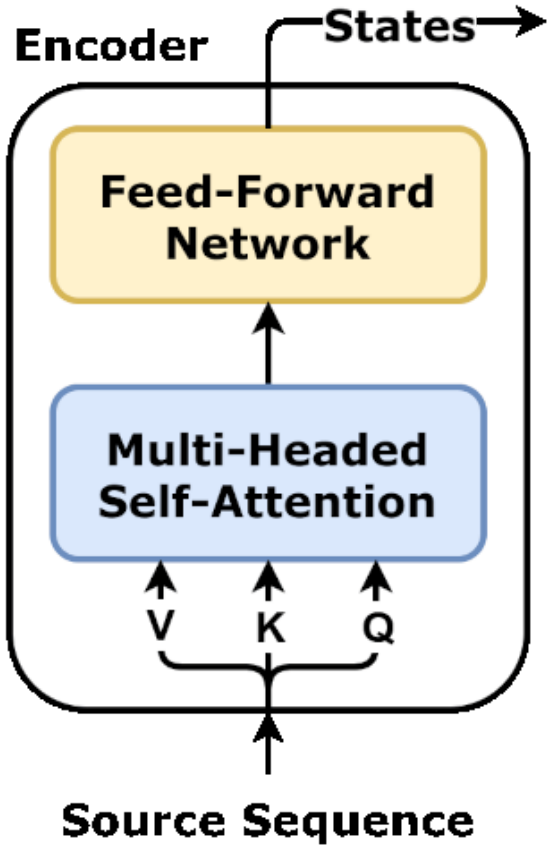


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder_block.png

One Decoder Layer

A decoder layer with masked self-attention and cross-attention:

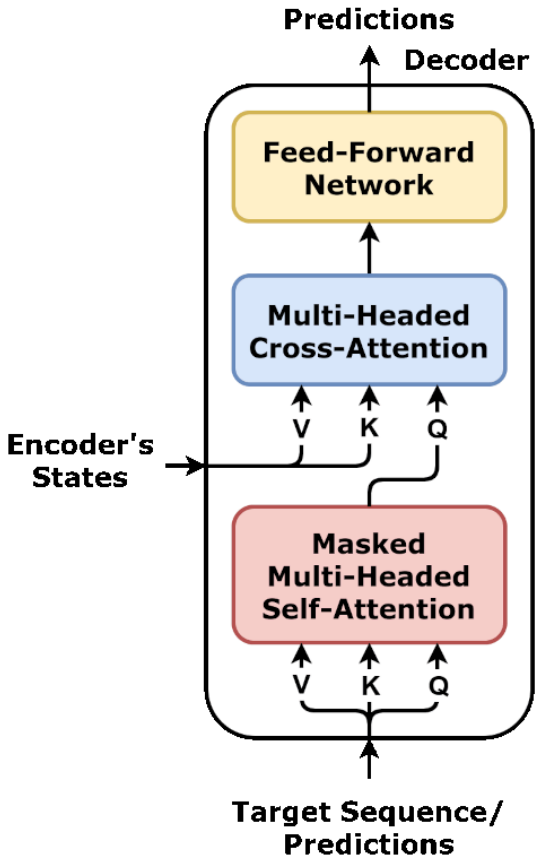


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_decoder_block.png

Encoder-Decoder Block

A single encoder-decoder block with self, cross and masked multi-head attention:

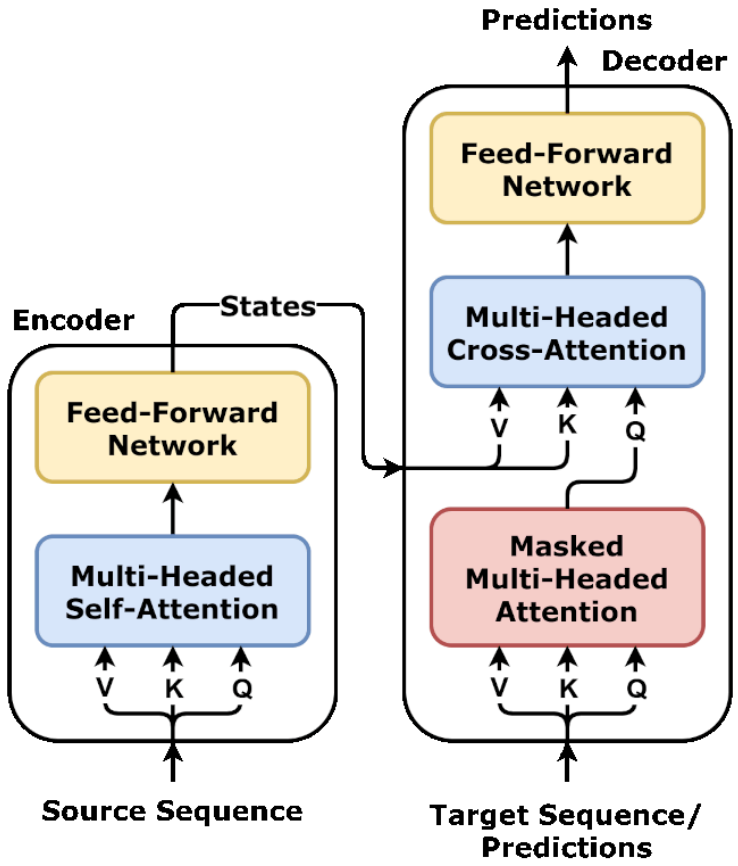


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder-decoder_block.png

Attention Revisited

In the attention chapter, we introduced **scaled dot-product attention** (Vaswani et al., 2017) with query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$ and value matrix $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, $\mathbf{K} \in \mathbb{R}^{m \times d_k}$, $\mathbf{V} \in \mathbb{R}^{m \times d_v}$, where n and m are the number of tokens in the query and key/value sequences, respectively — **each row corresponds to one token**
- The product $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times m}$ is a token-to-token score matrix: entry (i, j) measures how strongly token i attends to token j along the sequence
- **Self-attention**: $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ all originate from the same input sequence via learned projection matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

Self-Attention in the Encoder

In the encoder, every position attends to **all positions** in the input, including itself — no causal masking.

Given positioned input $\mathbf{Z} \in \mathbb{R}^{n \times d}$ (token embeddings with positional encodings added), where each of the n rows of $\mathbf{Z}$ corresponds to one token in the sequence:

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^V$$

$$\mathbf{O} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Since every position has full access to every other position, encoder self-attention is **bidirectional** — making the encoder well-suited for building rich contextual representations of the full input sequence.

Masked Self-Attention in the Decoder

During autoregressive generation the decoder must not attend to future tokens. A **causal mask** is applied before softmax (as introduced in the attention chapter):

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

After softmax, the $-\infty$ entries receive probability 0, so position i only aggregates information from positions up to and including i. This **unidirectional** (causal) attention supports both parallel training via teacher forcing and sequential autoregressive inference.

Attention in a Transformer

A single attention head with scaled dot product attention scores:

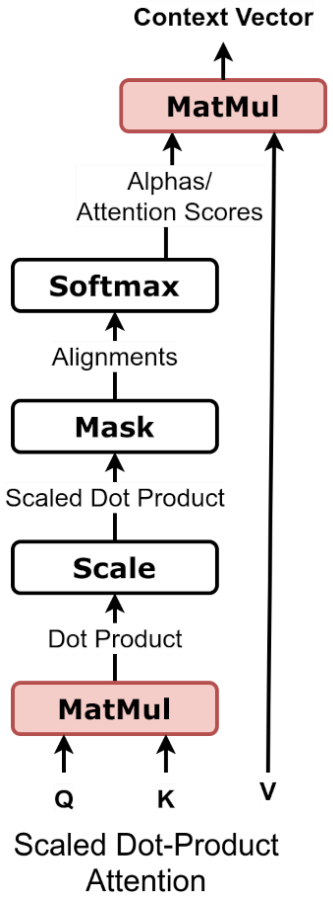


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_attention_block_diagram.png

Multi-Head Attention in a Transformer

Multi-head attention by concatenating independent heads:

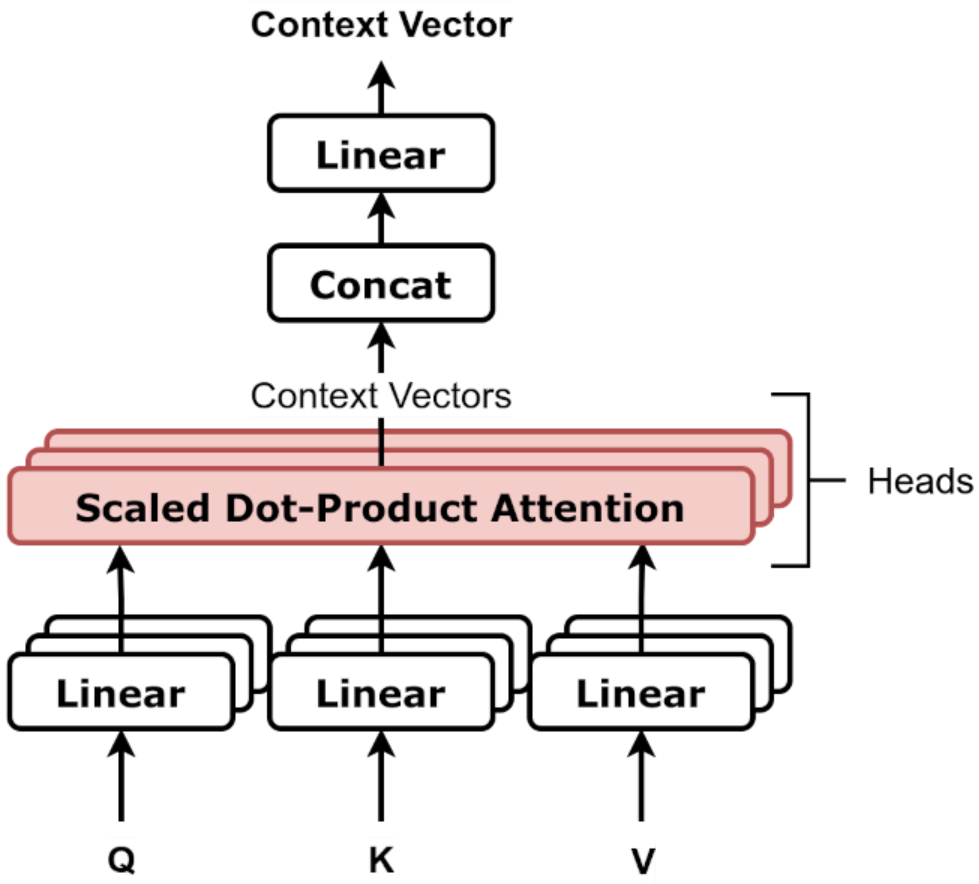


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Multiheaded_attention,_block_diagram.png

Cross-Attention in the Transformer

The decoder uses **cross-attention** to read from the encoder's output. Queries come from the decoder, keys and values come from the encoder:

$$\mathbf{Q} = \mathbf{H}^{(\text{dec})}\mathbf{W}^{Q}, \quad \mathbf{K} = \mathbf{H}^{(\text{enc})}\mathbf{W}^{K}, \quad \mathbf{V} = \mathbf{H}^{(\text{enc})}\mathbf{W}^{V}$$

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{H}^{(\text{dec})}$: decoder hidden states after masked self-attention
- $\mathbf{H}^{(\text{enc})}$: encoder hidden states from the top encoder layer

Each decoder position attends to **all encoder positions**, learning which parts of the source are most relevant for predicting the current target token.

Cross-Attention: Q, K, V Sources

Component	Source
Queries $\mathbf{Q}$	Decoder hidden states
Keys $\mathbf{K}$	Encoder hidden states
Values $\mathbf{V}$	Encoder hidden states

This is the same cross-attention mechanism introduced in the attention chapter — the key–value source is the encoder, while the query source is the decoder.

This allows the decoder to **condition on the full encoded source sequence** at every decoding step.

Encoder-Decoder Block

A single encoder-decoder block with self, cross and masked multi-head attention:

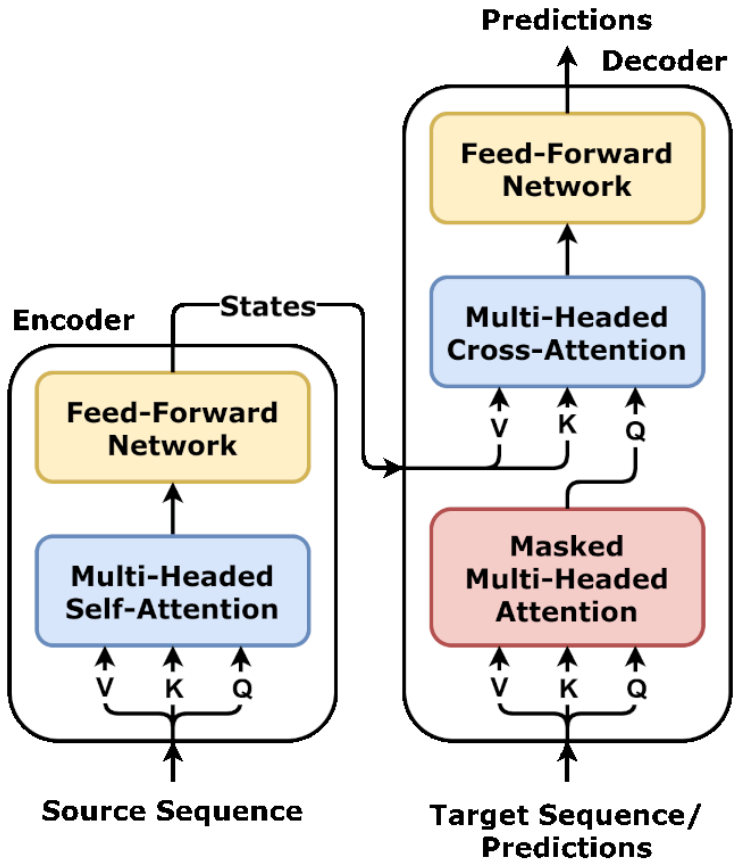


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder-decoder_block.png

Feed-Forward Sublayer

Each encoder and decoder layer contains a **position-wise feed-forward network** (FFN), applied **independently** to each sequence position:

$$\mathrm{FFN}(\mathbf{z}) = \mathbf{W}_2\,\sigma(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$$

where σ is a non-linear activation function (ReLU in the original paper).

- $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$ — the inner dimension is typically $d_{\text{ff}} = 4d$
- The FFN introduces **non-linearity** and expands the model's representational capacity
- The same weight matrices are shared across all positions in the sequence

Stacking Encoder and Decoder Layers

A single encoder or decoder layer provides limited representational depth. In the full Transformer, N layers are **stacked**:

- **Encoder stack**: N identical encoder layers applied in sequence; each layer refines the representations with self-attention and FFN
- **Decoder stack**: N identical decoder layers; each applies masked self-attention, cross-attention over the encoder output, and FFN

Stacked Encoder-Decoder Transformers

A single-layer and stacked encoder-decoder transformer side-by-side:

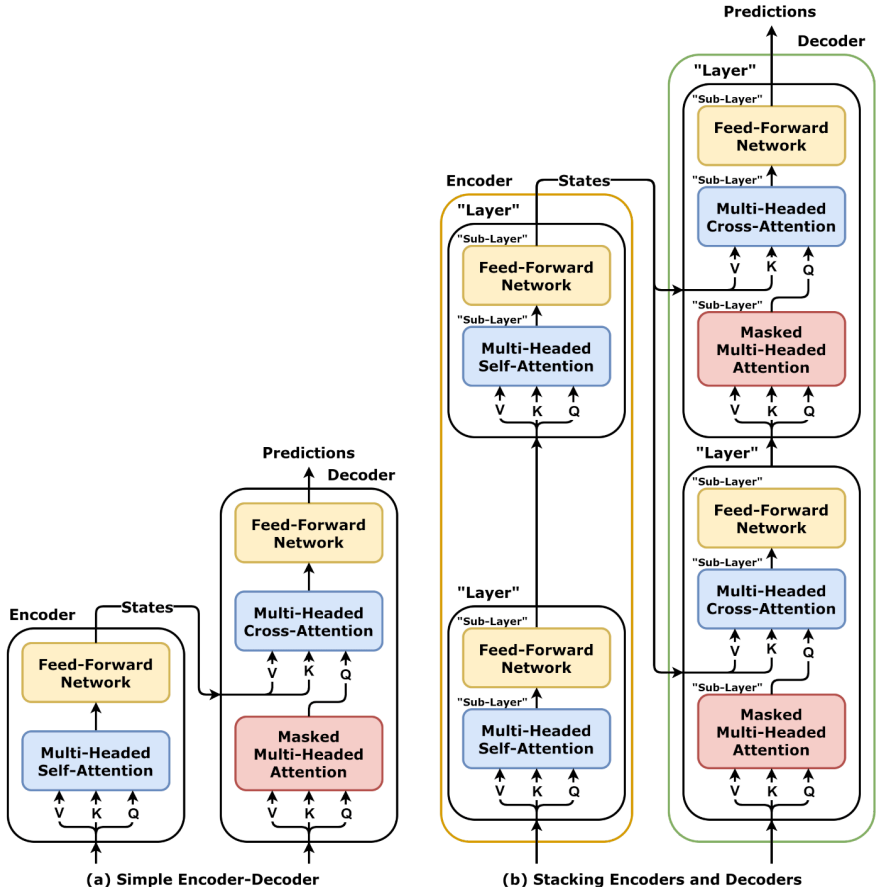


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_stacked_multilayers.png

Transformer Networks

A transformer with multi-layered encoder-decoder networks:

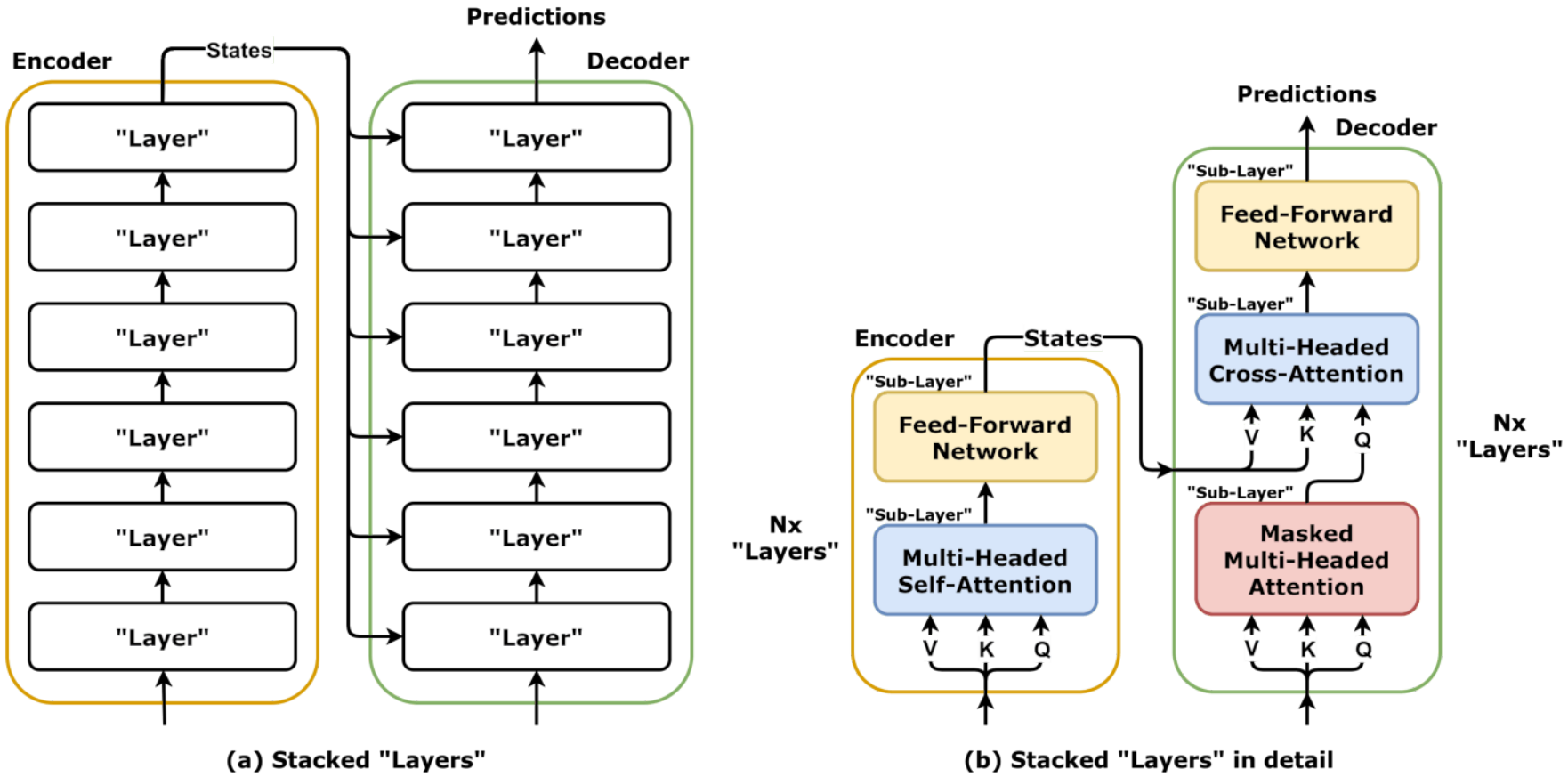


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_stacked_layers_and_sublayers.png

Transformer Networks

A standard transformer with embedding, positional encoding, encoder-decoder blocks and linear prediction:

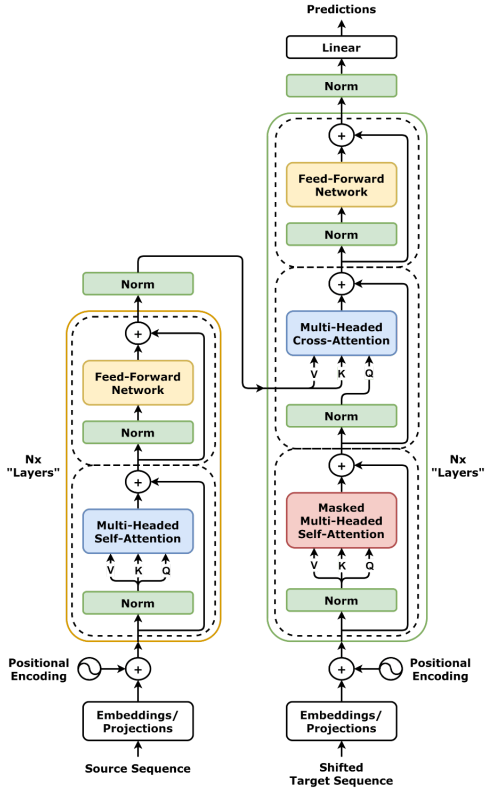


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_full_architecture.png

Encoder-Only Transformers

Encoder-only transformers use only the encoder stack. All positions attend to all others (bidirectional self-attention):

- Pre-trained with **masked language modeling** — randomly mask a fraction of tokens and predict them from the full context
- Well-suited for **understanding** tasks: classification, named entity recognition
- **Not suitable** for autoregressive generation — the model sees the full sequence at once

Applications: sentiment analysis, semantic search, text classification.

Decoder-Only Transformers

Decoder-only transformers use only the decoder stack with masked (causal) self-attention:

- Pre-trained with **next-token prediction** — predict token $t+1$ given tokens $1, \ldots, t$
- All positions attend only to previous positions — **unidirectional**
- Well-suited for **text generation**: language modeling, code completion, dialogue

At scale, decoder-only models can exhibit **emergent capabilities** such as few-shot learning, chain-of-thought reasoning when prompted appropriately, and in-context learning without weight updates.

Most modern large language models are decoder-only architectures.

GPT is Decoder-Only

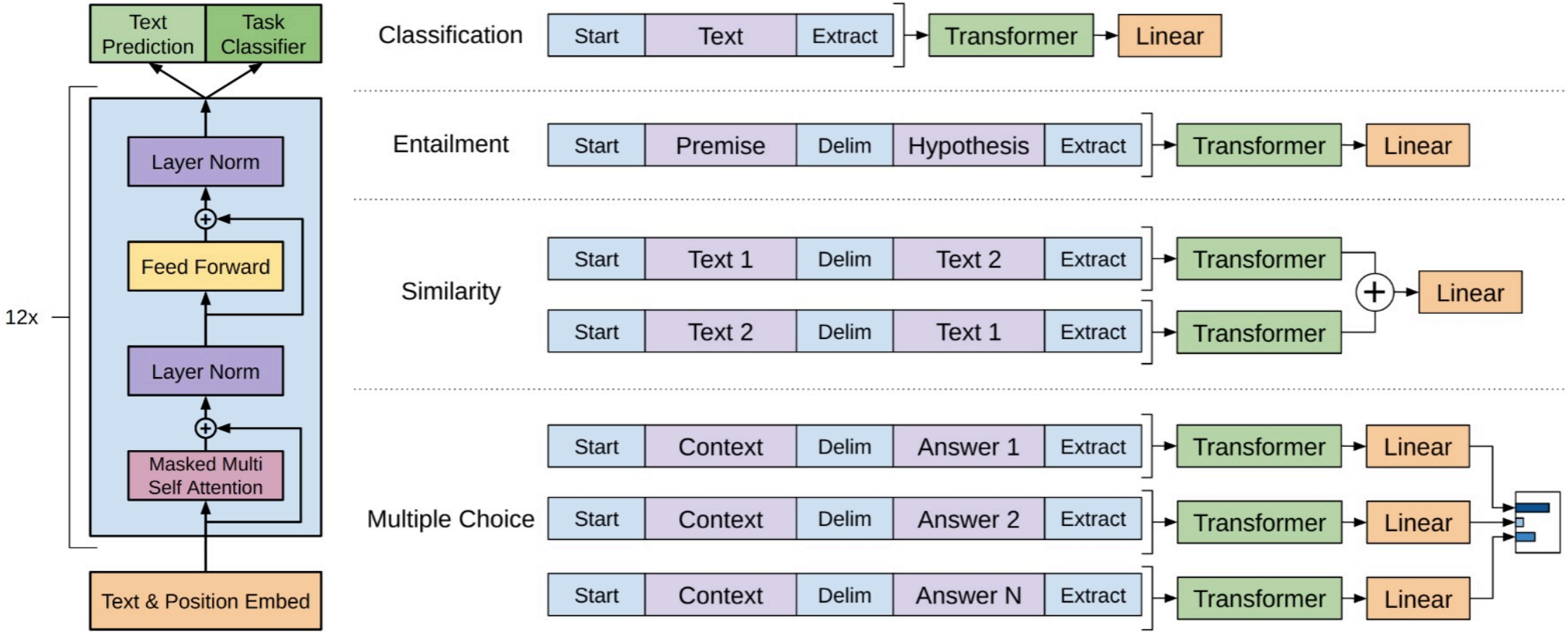


Image source: Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training.

Encoder-Decoder (Seq2Seq) Transformers

Encoder-decoder transformers use both stacks — the architecture introduced by Vaswani et al. (2017):

Well-suited for **sequence-to-sequence** tasks: machine translation, summarization, question generation.

The encoder processes the full source with bidirectional attention; the decoder generates the target autoregressively, attending to the encoder output via cross-attention at every step.

Autoregressive Language Modeling

The model predicts token t_i from a prefix representation $\mathbf{h}_{<i}$ computed from tokens $t_0, \ldots, t_{i-1}$:

$$\hat{P}(t_i \mid t_0, \ldots, t_{i-1}) = \mathrm{softmax}(\mathbf{W}_{\mathrm{out}}\, \mathbf{h}_{<i})$$

where $\mathbf{h}_{<i}$ explicitly denotes the representation of the prefix before token t_i, and $\mathbf{W}_{\mathrm{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}$

Training a Transformer

In language models, the target sequence is the input sequence shifted by one token:

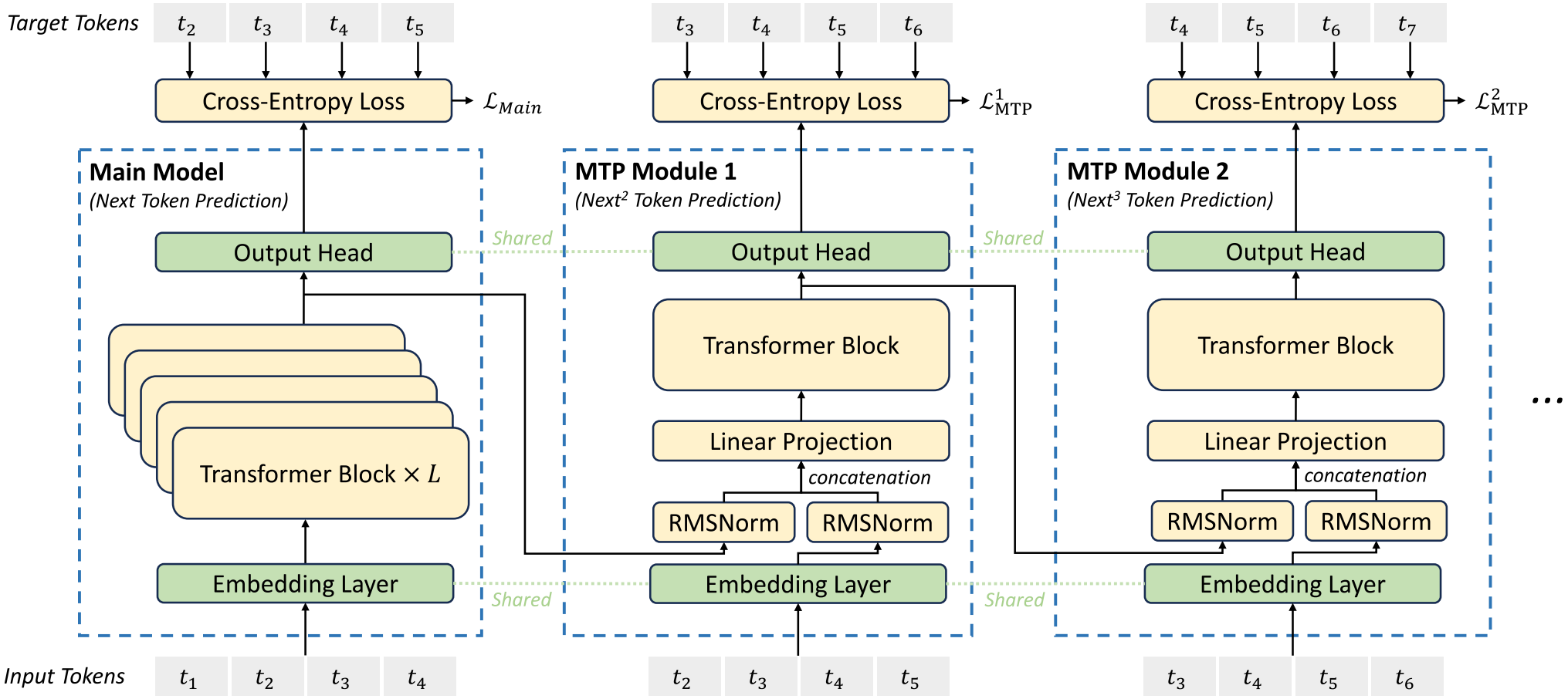


Image source: DeepSeek, MIT , via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Multi-Token_Prediction_(DeepSeek)_01.svg

Teacher Forcing

During training, the decoder receives the **ground-truth** tokens as input at each step — not its own previous predictions. This is called **teacher forcing**:

- At training step i, the decoder input is the ground-truth token t_{i-1}
- The decoder predicts a probability distribution over $\mathcal{V}$ for position i
- All positions are trained **in parallel** using the causal mask — enabling efficient GPU utilization

Teacher Forcing

At **inference** time, the model generates autoregressively:

1. Feed the start token `[BOS]`
2. Predict $\hat{t}_2 = \arg\max \hat{P}(\cdot \mid t_1)$ (or sample stochastically)
3. Append $\hat{t}_2$ and predict $\hat{t}_3$
4. Repeat until `[EOS]` or maximum length

Teacher forcing enables efficient parallel training while maintaining the autoregressive structure at inference.

Training Loss: Cross-Entropy

The training objective is to maximize the log-likelihood of the correct next token at every position. With a start token $t_0 = [\mathrm{BOS}]$ and ground-truth tokens $t_1, \ldots, t_n$, this is equivalent to minimizing the **cross-entropy loss**:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \log \hat{P}(t_i \mid t_0, \ldots, t_{i-1})$$

For a vocabulary of size $|\mathcal{V}|$, this can be written as a standard cross-entropy between the predicted distribution $\hat{\mathbf{y}}_i \in \mathbb{R}^{|\mathcal{V}|}$ and the one-hot target $\mathbf{y}_i$:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \mathbf{y}_i^\top \log \hat{\mathbf{y}}_i$$

LLaMa2 Training

Training loop of pretraining, finetuning and reinforcement learning in LLaMa2:

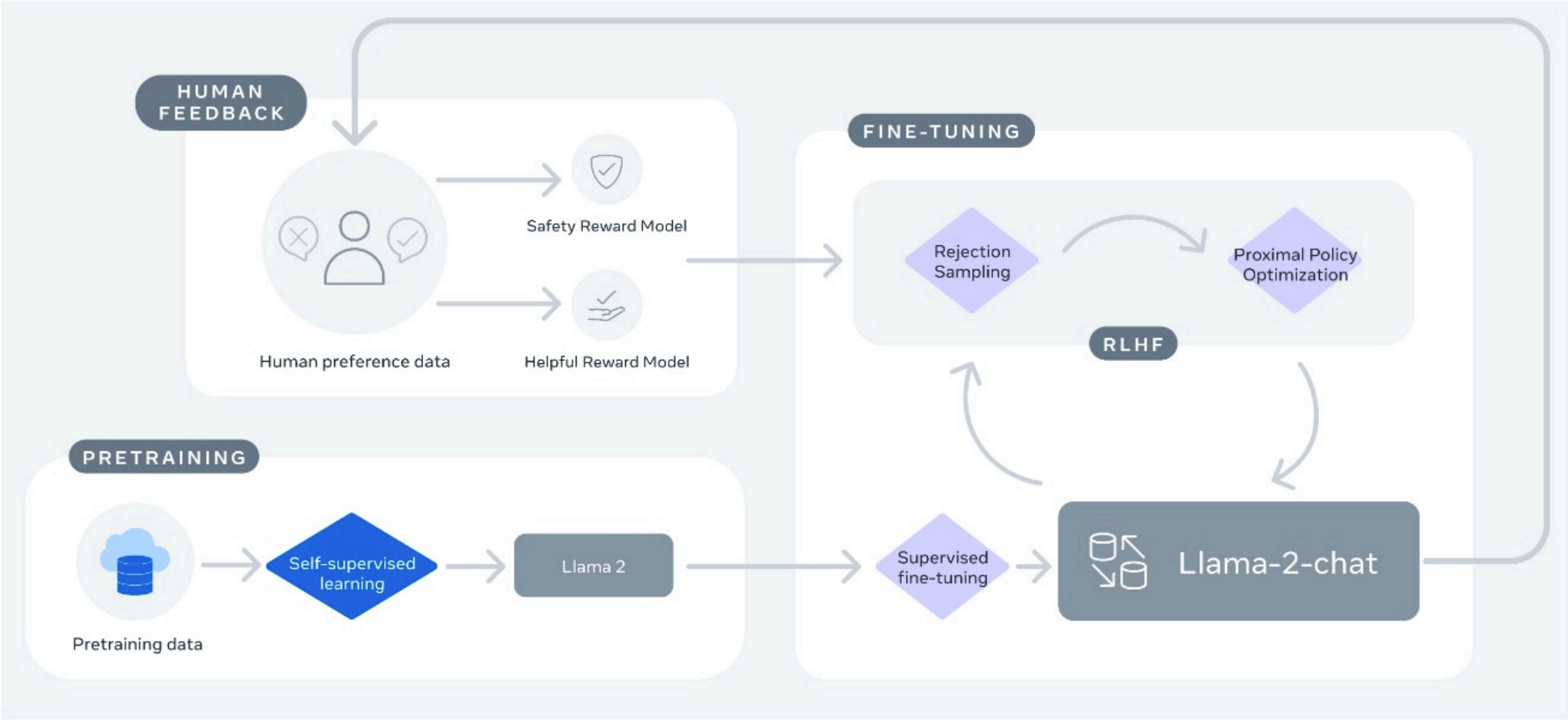


Image source: Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288.

Training at Scale

Large language models are trained on massive corpora with billions to trillions of tokens. Model sizes range from hundreds of millions to hundreds of billions of parameters.

Scaling laws (Kaplan et al., 2020) show that loss decreases as a predictable power law in both model size and dataset size — motivating ever-larger training runs.

Large-batch distributed training across hundreds or thousands of GPUs is standard.

Training at Scale

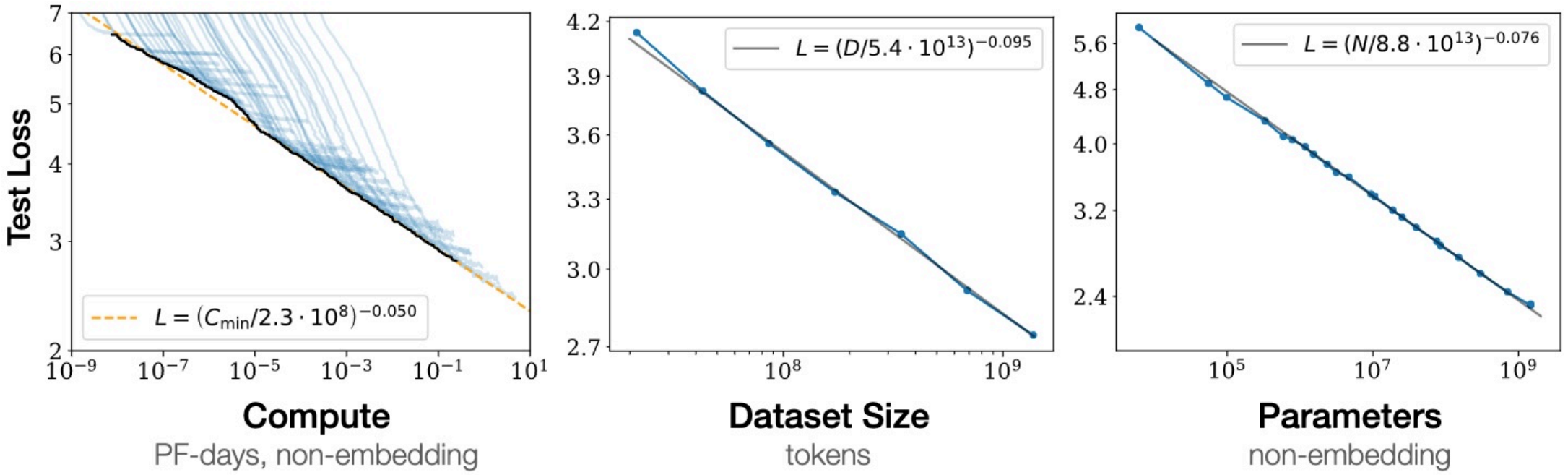


Image source: Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

The Context Window

The **context window** (or **context length**) is the maximum number of tokens the model can process in a single forward pass — the maximum value of n.

Everything the model can "see" and reason about at inference time must fit within this window:

- The system prompt
- The conversation history
- The current user input
- Any retrieved documents

If the input exceeds the context window, some tokens must be truncated, summarized or retrieved separately — tokens outside the current window are not processed in that forward pass.

Context Window: Memory and Compute

For a full-sequence forward pass, self-attention computes an $n \times n$ score matrix, so the attention score memory and compute costs grow **quadratically** with context length n:

$$\text{Attention score matrix size} = n^2 \text{ elements per head}$$

A model with a **16k context window** can attend to twice as many tokens as an 8k model, which requires approximately **four times** the memory for attention scores.

Context Window: Practical Meaning

What 8k vs. 16k means in practice:

Task	8k context	16k context
Short-document Q&A	✅ Fits easily	✅ Fits easily
Long article summarization	⚠️ May truncate	✅ Usually fits
Full book processing	❌ Requires chunking	❌ Requires chunking
Multi-turn dialogue	⚠️ History lost quickly	✅ Longer history preserved

The supported context window is determined by the architecture, positional encoding and training setup. Learned absolute positional embeddings impose a hard table size, while sinusoidal encodings can be evaluated at arbitrary positions but do not by themselves guarantee that models generalize to sequence lengths significantly beyond those seen during training.

Summary: The Transformer Architecture

The full Transformer pipeline from raw text to output prediction:

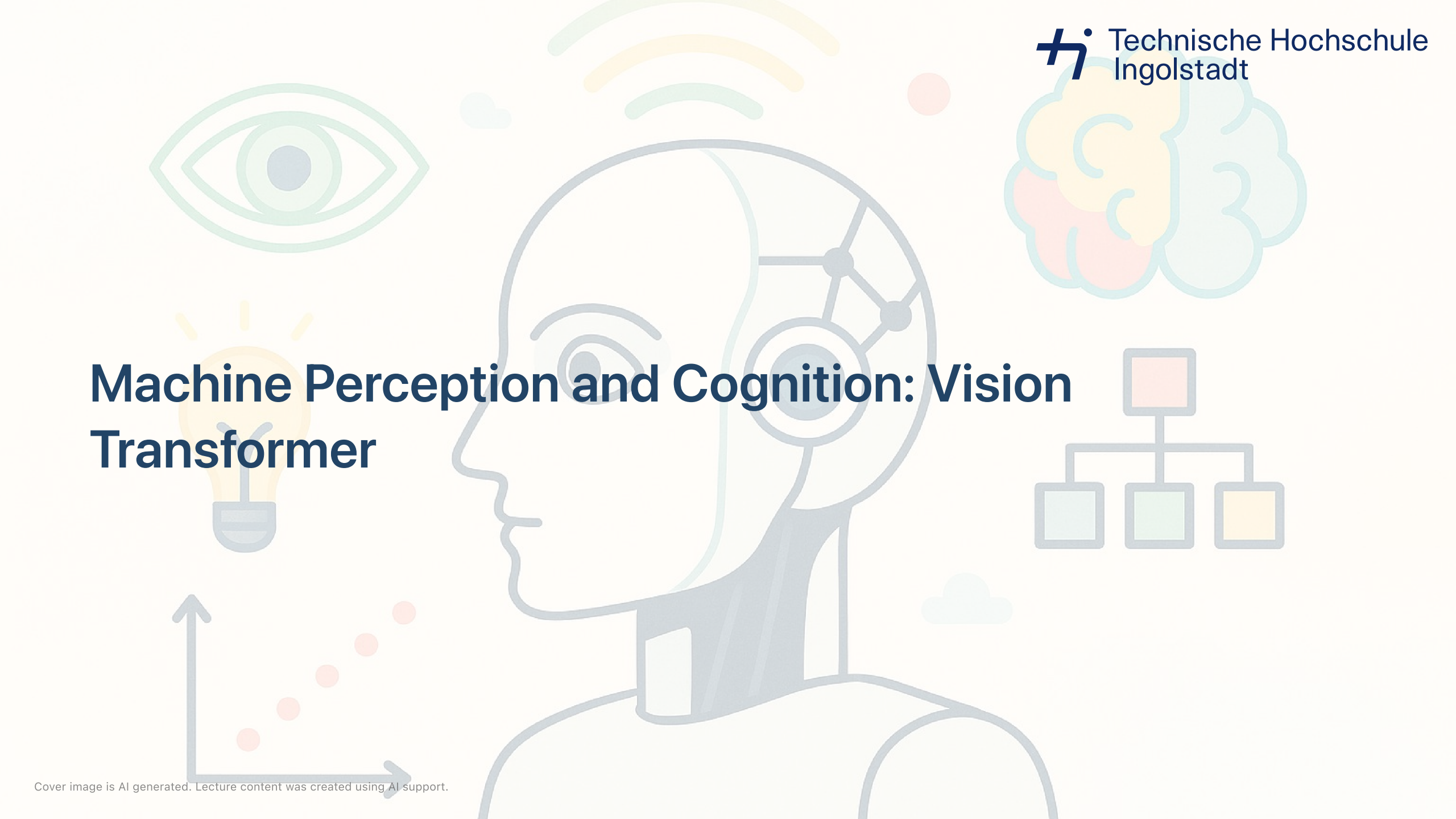
1. **Tokenize** raw text → sequence of token IDs using BPE or similar subword algorithm
2. **Embed** each token ID → dense vector via the embedding matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$
3. **Add positional encoding** → sinusoidal $PE(pos, i)$ added element-wise to each token embedding
4. **Encoder** (if present) → stack of bidirectional self-attention + FFN layers; produces key-value representations for the decoder
5. **Decoder** → stack of masked self-attention + cross-attention + FFN layers; generates output tokens autoregressively
6. **Output projection** → linear layer $\mathbf{W}_{\text{out}}$ + softmax → next-token probability distribution over $\mathcal{V}$

Summary: Why Transformers Work

Challenge	Transformer solution
Sequential bottleneck of RNNs	Parallel self-attention over all positions simultaneously
Vanishing gradients over long sequences	Direct attention connections between any two positions
Fixed-size context bottleneck in RNN seq2seq	Full encoder output accessible at every decoder step via cross-attention
Lack of positional information	Sinusoidal positional encoding $PE(pos, i)$ added to embeddings
Single attention pattern insufficient	Multi-head attention with H independent learned subspaces
Deep network training instability	Residual connections + layer normalization at every sublayer

Technische Hochschule Ingolstadt

Machine Perception and Cognition: Vision Transformer

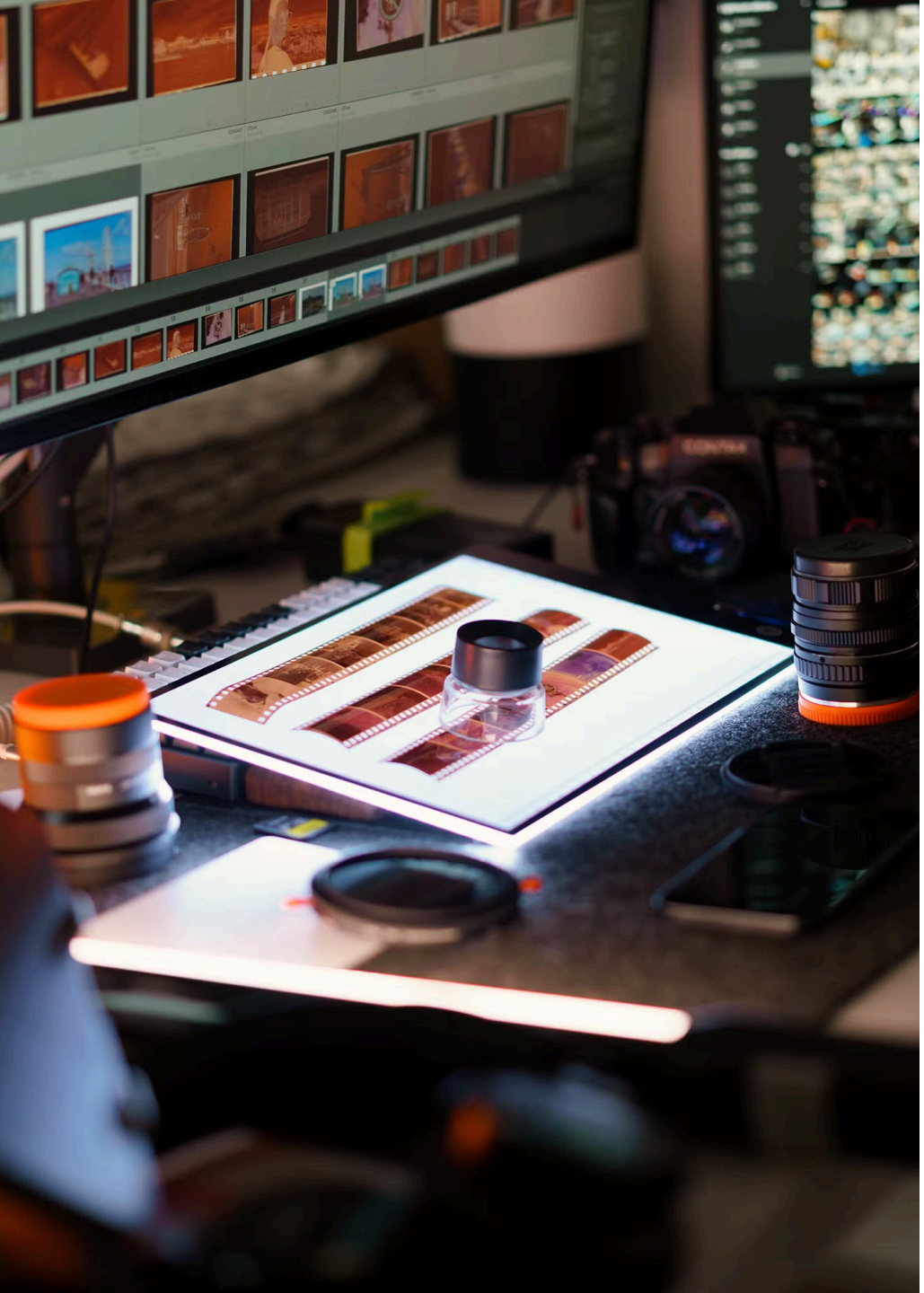


Cover image is AI generated. Lecture content was created using AI support.

Learning Goals

After this lecture you will be able to:

- Explain how images are converted into **patch sequences** for transformer processing
- Describe the full **Vision Transformer (ViT) architecture** — patch embedding, class token, positional encoding, encoder blocks, and classification head
- Explain how **self-attention operates on image patches** and why attention maps are interpretable
- Compare the inductive biases of **CNNs and ViT**
- Describe **five self-supervised and supervised pre-training strategies** for ViT: supervised classification, masked autoencoder, SimCLR, CLIP, and DINOv2



Vision Transformer

So how do we go from a transformer processing text to a transformer processing images?

Image source: https://www.pexels.com/photo/professional-photography-workspace-with-film-development-tools-37204672/

From Tokens to Patches

The Transformer we studied in the previous chapter expects a **sequence of vectors** as input:

$$\mathbf{Z} = (\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_n) \in \mathbb{R}^{n \times d}$$

For text, each $\mathbf{z}_i$ is a token embedding. The core question for visual data is:

> **How do we turn an image into a sequence of vectors?**

From Tokens to Patches

Option 1 — pixel sequence: flatten the image $H \times W \times C$ into one long sequence. For a 224×224 RGB image this gives $150\,528$ tokens — prohibitively long, because self-attention computes a score between every pair of tokens, so memory and compute grow with the square of the sequence length.

Option 2 — patch sequence: split the image into non-overlapping patches and treat each patch as one token. A 16×16 patch on a 224×224 image gives only 196 tokens. This is the key idea of the **Vision Transformer (ViT)**.

Image Patches as Tokens

An image $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ is divided into N non-overlapping patches, each of size $P \times P$:

$$N = \frac{H \cdot W}{P^2}$$

Each patch $\mathbf{p}_i \in \mathbb{R}^{P^2 C}$ is flattened into a vector and linearly projected to the model dimension d:

$$\mathbf{z}_i^{(0)} = \mathbf{p}_i \mathbf{E}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 C) \times d}$$

The projection matrix $\mathbf{E}$ is the **patch embedding layer** — a learned linear map trained jointly with the rest of the model.

For $H = W = 224$, $P = 16$, $C = 3$: $N = 196$ patches, each of dimension $P^2 C = 768$ before projection.

Vision Transformer

Principle of a vision transformer, processing an image as a sequence of patches:

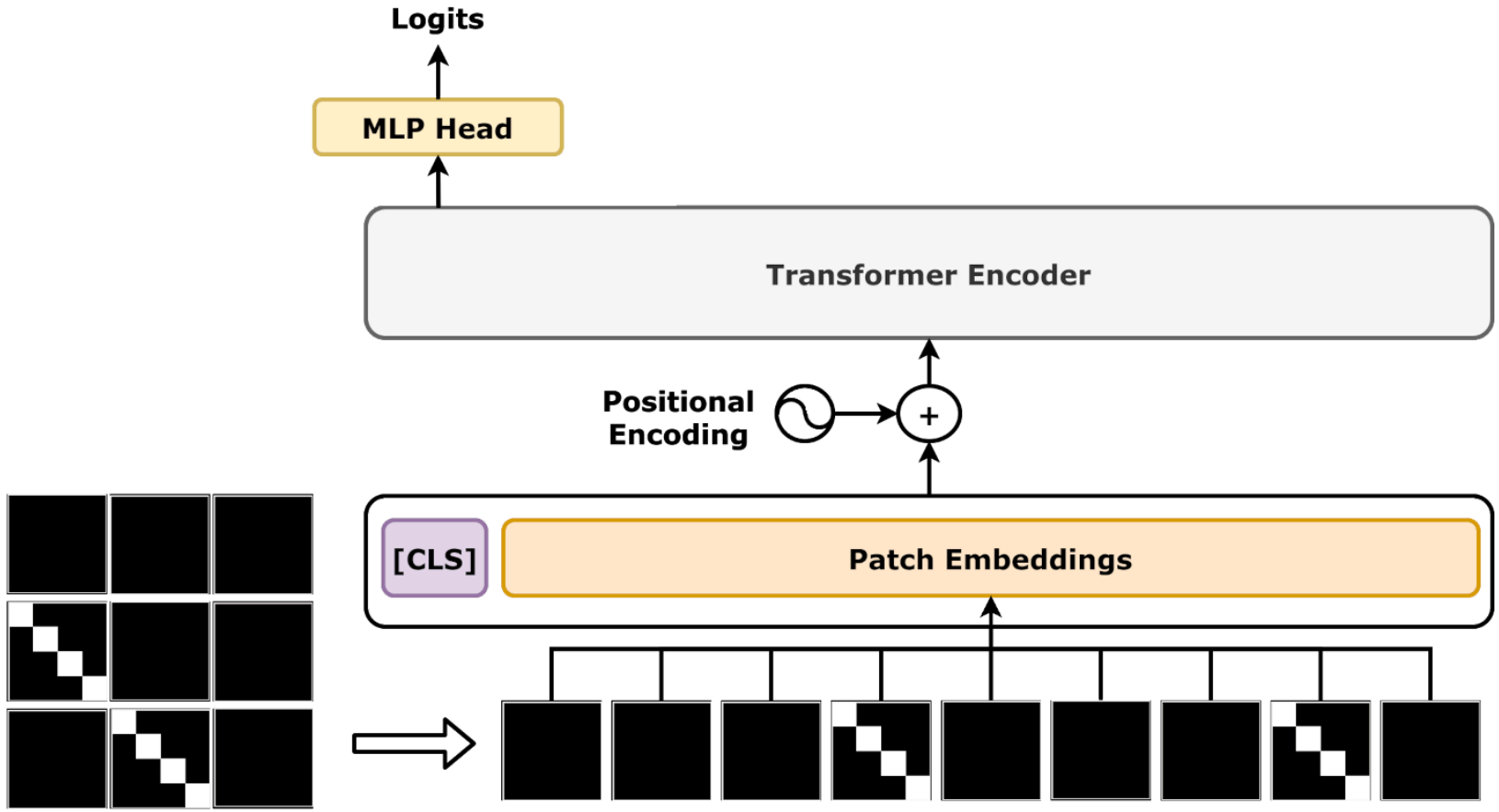


Image source: Daniel Voigt Godoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Vision_Transformer.png

Vision Transformer

Vision transformer with a dog+cat image as input:

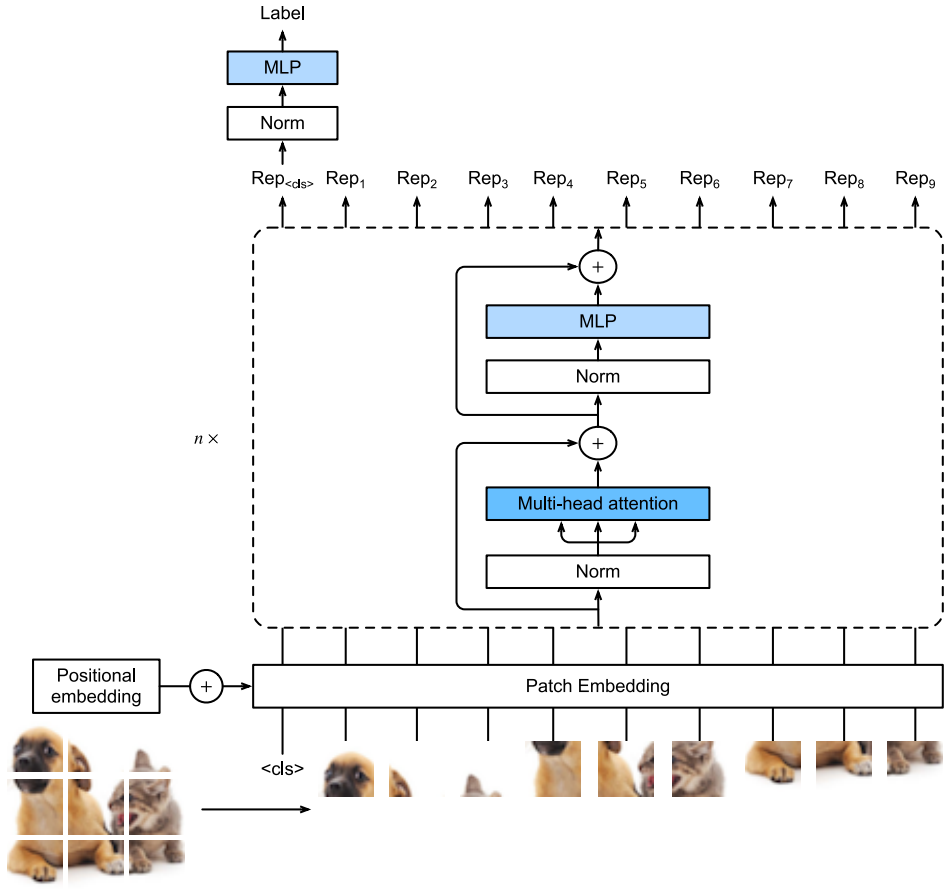


Image source: Zhang, Aston and Lipton, Zachary C. and Li, Mu and Smola, Alexander J., CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Vision_Transformer.svg

The Class Token

ViT prepends a special learnable **class token** $\mathbf{z}_{\text{cls}} \in \mathbb{R}^d$ to the patch sequence:

$$\mathbf{Z}^{(0)} = \left[\mathbf{z}_{\text{cls}},\ \mathbf{z}_1^{(0)},\ \mathbf{z}_2^{(0)},\ \ldots,\ \mathbf{z}_N^{(0)}\right] \in \mathbb{R}^{(N+1)\times d}$$

Purpose of the class token:

- It is not tied to any specific patch — it acts as a global summary token
- After L transformer encoder layers, the output at position 0 (i.e., $\mathbf{z}_{\text{cls}}^{(L)}$) aggregates global image information through attention
- A classification head (MLP) is applied to $\mathbf{z}_{\text{cls}}^{(L)}$ to predict the class label

The class token learns to attend to the most discriminative patches across all layers.

Positional Encoding for Patches

Self-attention is permutation-equivariant — without positional information, the model cannot distinguish between different spatial arrangements of patches.

ViT adds **learnable 1D positional embeddings** $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times d}$ to the patch embeddings:

$$\mathbf{Z}^{(0)} \leftarrow \mathbf{Z}^{(0)} + \mathbf{E}_{\text{pos}}$$

Each of the $N + 1$ positions (class token + N patches) gets its own learned position vector.

Why 1D instead of 2D? The original ViT paper found that 1D positional embeddings perform comparably to 2D sinusoidal encodings — the model learns spatial structure from data rather than from the encoding scheme.

Vision Transformer

Vision transformer as in the original paper:

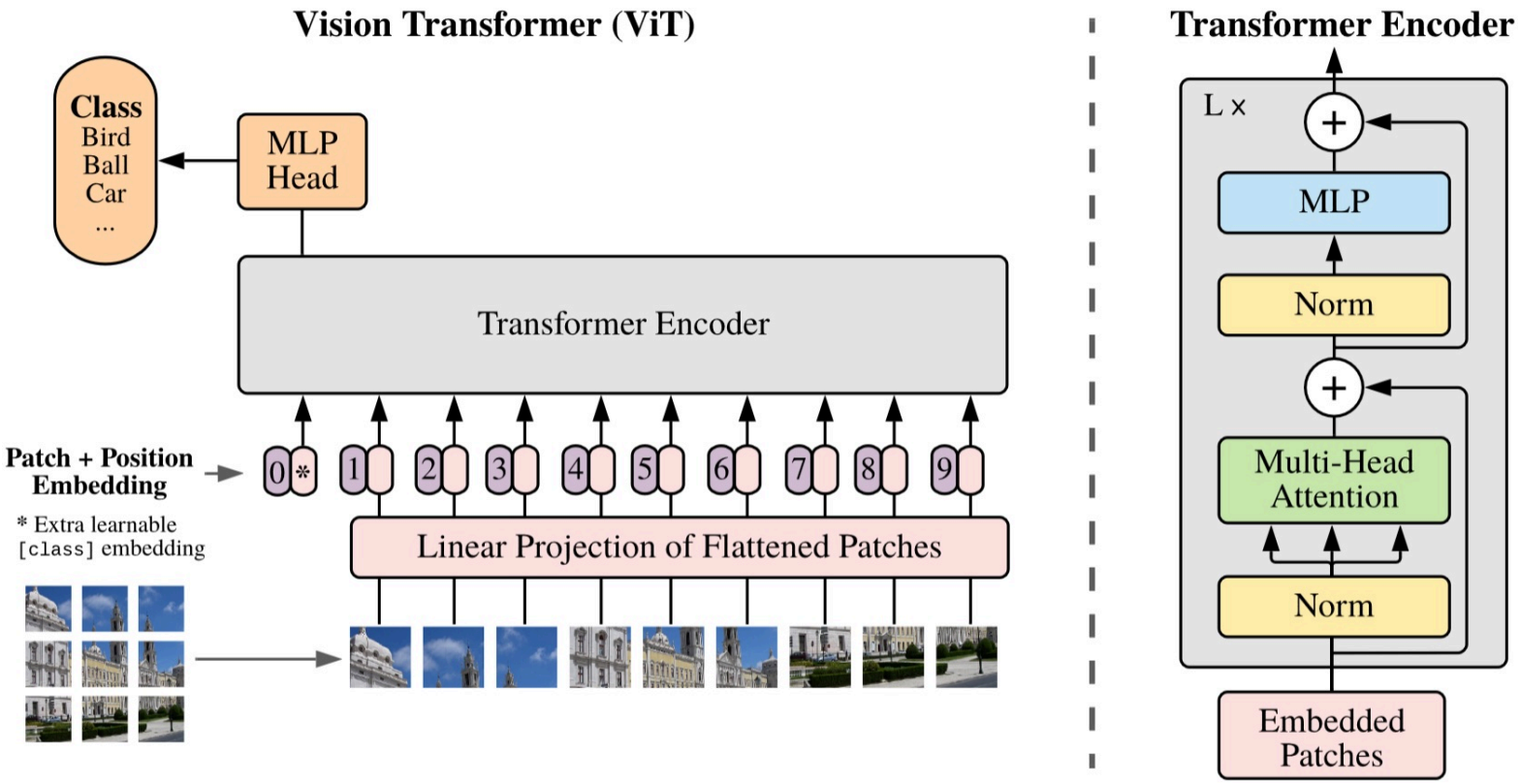


Image source: Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.

ViT Classification Head

After L encoder layers, the class token representation $\mathbf{z}_{\text{cls}}^{(L)} \in \mathbb{R}^d$ is passed to an **MLP head**:

$$\hat{\mathbf{y}} = \text{softmax}\Big(\mathbf{W}_2\, \sigma(\mathbf{W}_1\, \mathbf{z}_{\text{cls}}^{(L)} + \mathbf{b}_1) + \mathbf{b}_2\Big)$$

During pre-training, ViT uses a classification head (linear or MLP, as shown here). During fine-tuning, this head is replaced with a task-specific head, typically a new linear layer.

Alternative to class token — global average pooling:
Instead of using $\mathbf{z}_{\text{cls}}^{(L)}$, some ViT variants average the N patch tokens after the last encoder layer:

$$\mathbf{z}_{\text{gap}}^{(L)} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{z}_i^{(L)}$$

Class Token Aggregation

The class token plays a central role in information aggregation:

Layer 0 (input): $\mathbf{z}_{\text{cls}}^{(0)}$ is a randomly initialized learnable vector — it contains no image information yet.

After each layer ℓ: the class token aggregates information from all patch tokens through cross-token attention. Its representation is progressively refined.

After layer L: $\mathbf{z}_{\text{cls}}^{(L)}$ has attended to all patches across L rounds of multi-head self-attention — it encodes a rich, global summary of the image that is used for classification.

This differs fundamentally from CNNs, where global pooling aggregates spatial features only at the final layer.

ViT Model Variants

The original ViT paper (Dosovitskiy et al., 2020) defines three standard model sizes:

Variant	Layers L	Hidden dim d	Heads	Params
ViT-B (Base)	12	768	12	86M
ViT-L (Large)	24	1024	16	307M
ViT-H (Huge)	32	1280	16	632M

CNN vs. ViT: Inductive Biases

CNNs build in strong spatial inductive biases:

- **Translation equivariance**: applying a filter at different positions yields the same response
- **Locality**: filters only see a local receptive field at each step
- These biases are effective when training data is limited

CNN vs. ViT: Inductive Biases

ViT has almost no spatial inductive bias:

- Self-attention has a **global receptive field** from the first layer — every patch can attend to every other patch
- Only 2D structure used: patch extraction and positional encoding
- Without inductive bias, ViT needs much more data to learn spatial relationships

Consequence: ViT underperforms CNNs on small datasets but surpasses them when pre-trained at scale — the model learns spatial structure from data.

Self-Attention on Image Patches

In each ViT encoder layer, multi-head self-attention is applied to the sequence of $N+1$ tokens (patches + class token):

For each attention head h with projection matrices $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{d \times d_k}$:

$$\mathbf{Q}_h = \mathbf{Z}\mathbf{W}_h^Q, \quad \mathbf{K}_h = \mathbf{Z}\mathbf{W}_h^K, \quad \mathbf{V}_h = \mathbf{Z}\mathbf{W}_h^V$$

$$\mathbf{A}_h = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{(N+1)\times(N+1)}$$

The (i, j)-entry of $\mathbf{A}_h$ is the **attention weight** from patch i to patch j — how much patch i attends to the content of patch j.

No causal masking is applied — all patches can attend to all others.

Multi-Head Attention: Different Perspectives

With h attention heads, each head learns a **different attention pattern**:

- Some heads may focus on **local texture** within neighboring patches
- Other heads capture **long-range semantic relationships** (e.g., relating a head to a body)
- Some heads specialize in **background suppression**

This multi-head mechanism allows the model to jointly attend to information from different representation subspaces at different positions — directly analogous to how different CNN filters detect different features.

Attention Maps in ViT

The attention weight matrix $\mathbf{A}_h^{(\ell)} \in \mathbb{R}^{(N+1)\times(N+1)}$ at layer ℓ can be visualized as a **spatial attention map** over the patch grid.

For the class token (row 0), the weights across the N patch positions reveal which patches the class token directly attends to at that layer. Averaging over all n_h heads gives a single per-layer map:

$$\mathbf{A}^{(\ell)} = \frac{1}{n_h}\sum_{h=1}^{n_h}\mathbf{A}_h^{(\ell)}, \qquad a_{0,j}^{(\ell)} = [\mathbf{A}^{(\ell)}]_{0,j}, \quad j = 1, \ldots, N$$

Attention Maps in ViT

Limitation: a single-layer map captures only *direct* attention at layer ℓ. It ignores how information propagated through earlier layers — the class token may attend to a patch that itself attended to relevant regions in layer $\ell - 1$.

Residual Connections as Part of Attention

When visualizing attention, do not forget the residual connections!

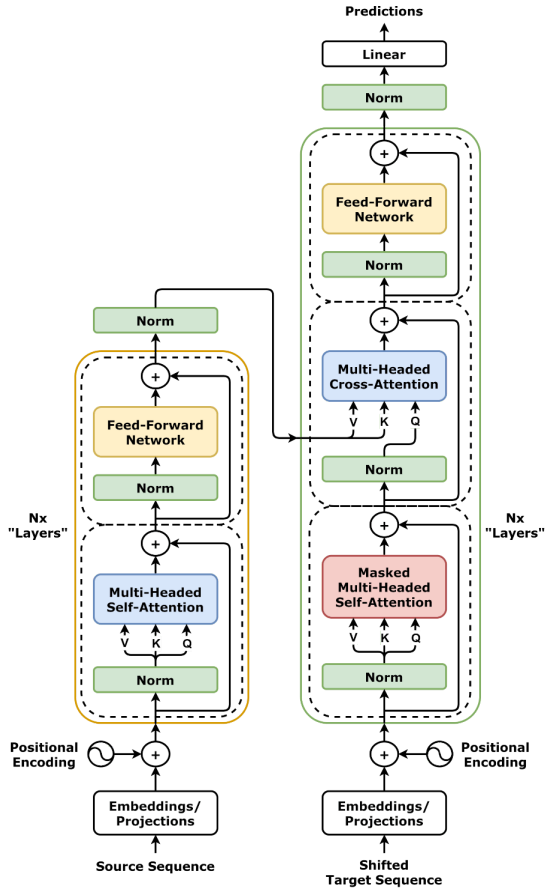


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_full_architecture.png

Attention Rollout

Attention Rollout (Abnar & Zuidema, 2020) addresses this by propagating attention *recursively* from the last layer back to the input, accounting for the residual connection in each block.

Residual connection as identity attention: Each transformer block applies attention and then adds the original input via a skip connection. For token representations this means:

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell)}\,\mathbf{Z}^{(\ell-1)} + \mathbf{I}\,\mathbf{Z}^{(\ell-1)} = \left(\mathbf{A}^{(\ell)} + \mathbf{I}\right)\mathbf{Z}^{(\ell-1)}$$

Attention Rollout

The identity $\mathbf{I}$ models the residual path — every token always "attends to itself" through the skip connection. Because both $\mathbf{A}^{(\ell)}$ (rows sum to 1) and $\mathbf{I}$ (rows sum to 1) contribute, the combined matrix has rows summing to 2. Row-normalizing restores a proper attention distribution:

$$\tilde{\mathbf{A}}^{(\ell)} = \frac{\mathbf{A}^{(\ell)} + \mathbf{I}}{2} = 0.5\,\mathbf{A}^{(\ell)} + 0.5\,\mathbf{I}$$

Starting from $\hat{\mathbf{A}}^{(0)} = \mathbf{I}$, the rolled-out attention is built recursively for $\ell = 1, \ldots, L$:

$$\hat{\mathbf{A}}^{(\ell)} = \tilde{\mathbf{A}}^{(\ell)} \cdot \hat{\mathbf{A}}^{(\ell-1)}$$

Row 0 of the final matrix $\hat{\mathbf{A}}^{(L)}$ gives the **effective contribution** of each input patch to the class token after L layers of transitive attention flow.

Attention Rollout

Upscaling: extract $\hat{\mathbf{A}}^{(L)}_{0,\,1:N}$ (N scalar weights), reshape to $\frac{H}{P} \times \frac{W}{P}$, and bilinearly upsample to $H \times W$ to obtain a full-resolution interpretability map.

Abnar, S. & Zuidema, W. (2020). Quantifying Attention Flow in Transformers. Proceedings of ACL 2020. arXiv:2005.00928

Attention in a Vision Transformer

Some examples with input image left and attention map right:

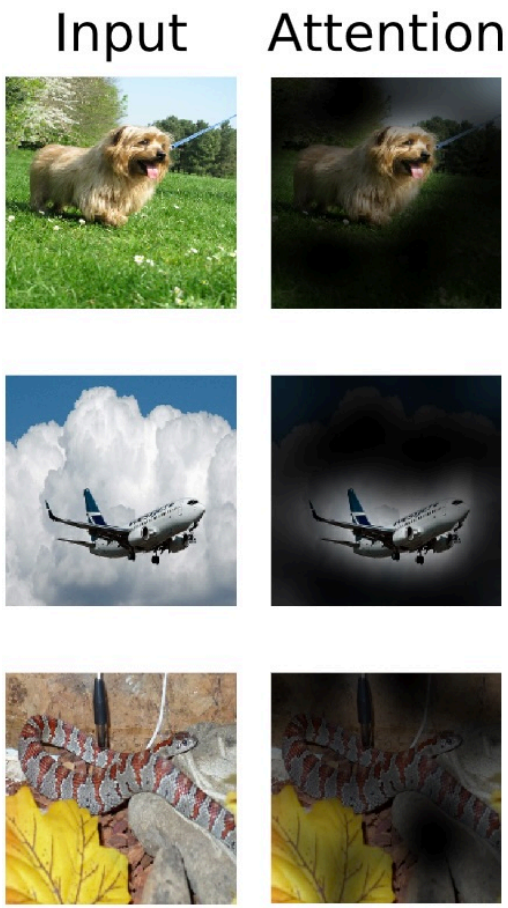


Image source: Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.

Attention in a Vision Transformer

Some more examples:

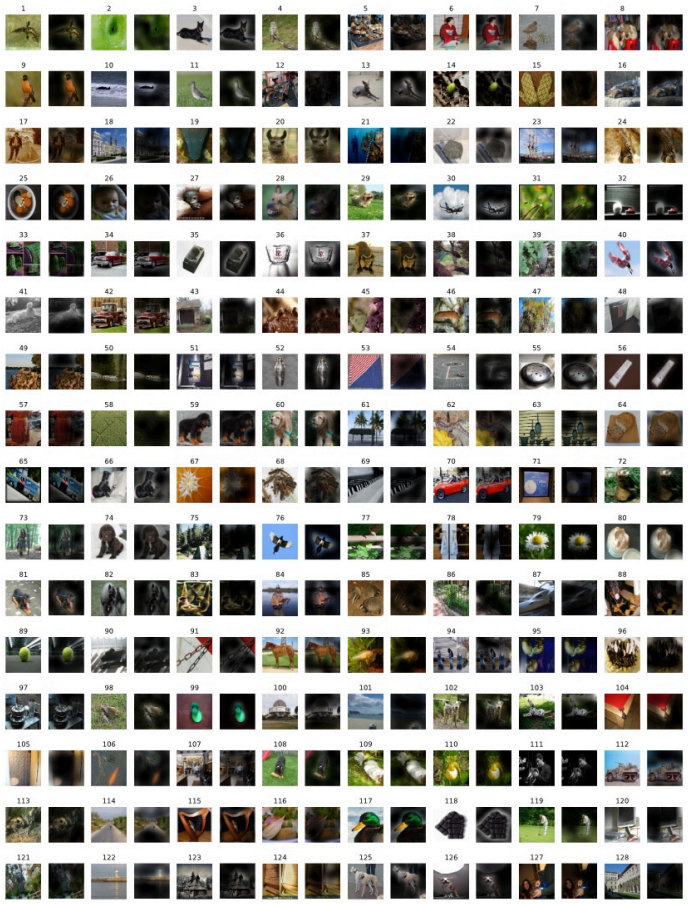


Image source: Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.

Pre-Training a Vision Transformer

How can we pre-train a vision transformer?

Image source: https://www.pexels.com/de-de/foto/fragezeichen-illustration-356079/

Why Pre-Training?

Training a large ViT from scratch on a small task-specific dataset overfits quickly due to the **lack of inductive bias**.

Pre-training first trains the model on a large dataset or with a self-supervised objective, then **fine-tunes** the resulting model on a smaller downstream task:

$$\underbrace{\text{Large data / self-supervised}}_{\text{pre-training}} \xrightarrow{\text{fine-tune}} \underbrace{\text{small labeled task}}_{\text{downstream}}$$

Supervised Pre-Training on ImageNet

The most straightforward pre-training: train ViT with a **cross-entropy classification loss** on ImageNet:

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

where K is the number of classes (1000 for ImageNet-1k, 21,841 for ImageNet-21k).

Class probabilities $\hat{y}_k$ are estimated by placing a classification head (typically an MLP during supervised pre-training) on the final class-token representation.

Supervised Pre-Training: Transfer

After ImageNet pre-training, the ViT classification head is replaced and the model is fine-tuned on a downstream task:

1. Remove the pre-training classification head
2. Attach a task-specific head (e.g., a linear layer for a new classification task)
3. Fine-tune the full model (or just the head) with a small learning rate

Masked Autoencoder

The MAE encoder and decoder are both stacks of transformer encoder blocks. The decoder is used only for reconstruction during pre-training, making the concept similar to a denoising autoencoder:

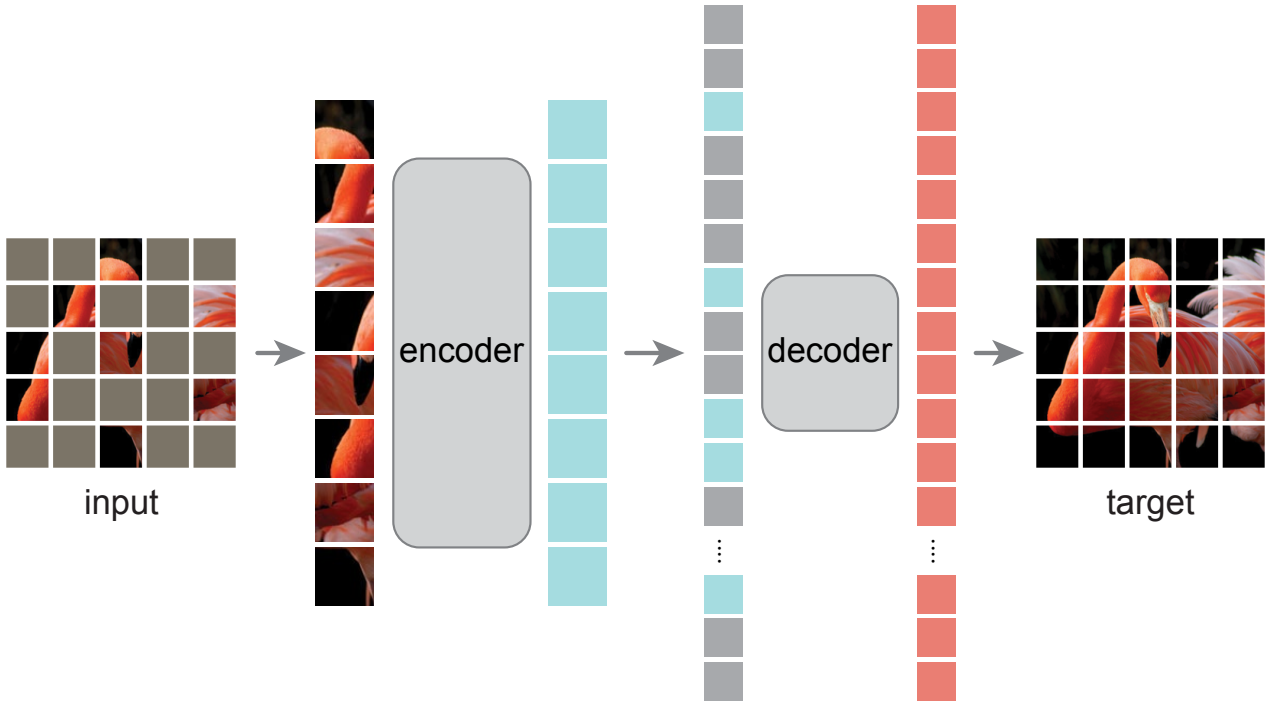


Image source: Kaiming He and Xinlei Chen and Saining Xie and Yanghao Li and Piotr Dollár and Ross Girshick, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Masked_Autoencoder.svg

Masked Autoencoder: Core Idea

The Masked Autoencoder (MAE, He et al., 2022) is a **self-supervised pre-training** method that learns useful visual representations by solving a reconstruction task:

1. **Mask** a large fraction (typically 75%) of the N image patches at random
2. **Encode** only the **visible** (unmasked) patches using the full ViT encoder — this is the key efficiency gain
3. **Decode** all N patch positions (visible + masked) using a **lightweight decoder** ViT, predicting the pixel content of the masked patches

Since the encoder only processes $\sim 25\%$ of patches, MAE pre-training is computationally efficient despite the large ViT backbone.

MAE: Asymmetric Encoder–Decoder

Encoder (large, full ViT):

- Receives only the visible patch tokens (no mask tokens)
- Produces rich representations of visible patches
- This is the part that is used after pre-training

Decoder (small, lightweight ViT):

- Receives encoded visible patches + learnable mask tokens at masked positions
- Reconstructs the pixel content of all N patches
- Discarded after pre-training — only the encoder weights are kept

The asymmetry means the heavy encoder never processes uninformative mask tokens.

MAE: Reconstruction Objective

MAE predicts the **normalized pixel values** of the masked patches.

Each patch is divided into $P \times P$ pixels. The decoder outputs a vector of size P^2C per patch position, compared against the ground-truth patch pixels.

The training loss is the **mean squared error (MSE)** computed only over the masked patches:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \left\| \hat{\mathbf{p}}_i - \mathbf{p}_i \right\|^2$$

where $\mathcal{M}$ is the set of masked patch indices, $\mathbf{p}_i$ is the normalized ground-truth patch, and $\hat{\mathbf{p}}_i$ is the decoder prediction.

Masked Autoencoder Examples

Examples from the ImageNet dataset:

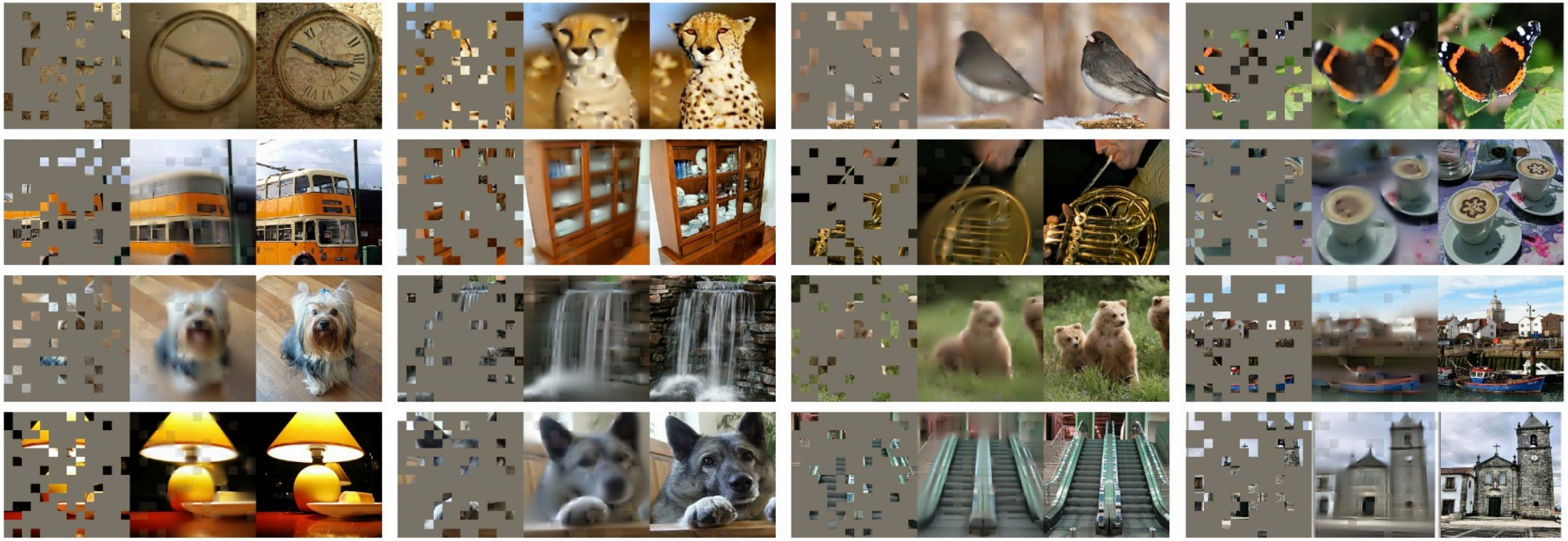


Image source: He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). Masked autoencoders are scalable vision learners. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 16000-16009).

Masked Autoencoder Examples

Examples from the COCO dataset:

Image source: He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). Masked autoencoders are scalable vision learners. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 16000-16009).

MAE: Why High Masking Ratio?

Masking 75% of patches is key:

- Low masking → reconstruction is too easy; patches can be reconstructed from adjacent visible patches by simple interpolation
- High masking → the model must **understand global image structure** to fill in large missing regions
- The task is genuinely hard only when context is sparse: the model must learn meaningful semantic representations, not just local texture matching

This high masking ratio also enables the encoder to run on only ~25% of the patches per image, dramatically reducing compute during pre-training.

SimCLR Transformations

Transformations $\mathcal{T}$ used in the original SimCLR paper:

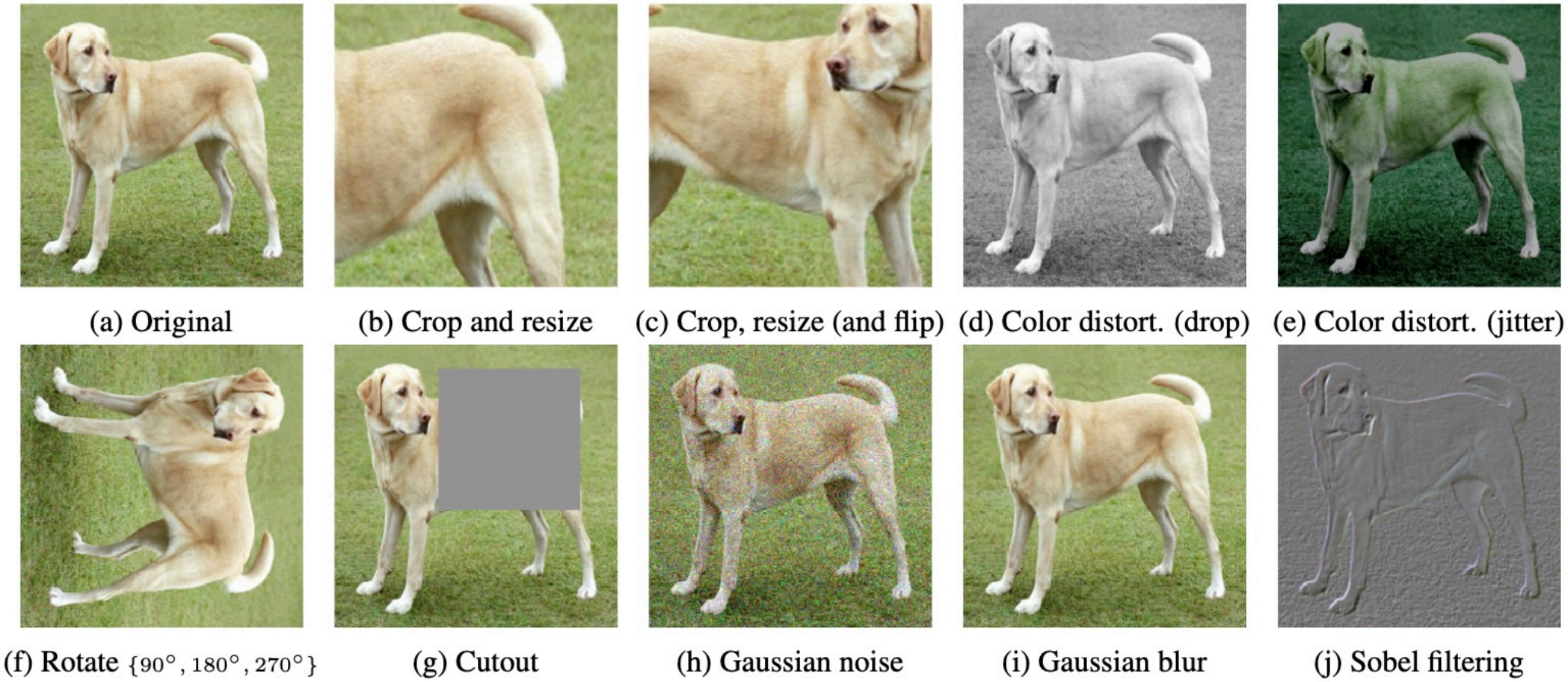


Image source: Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020, November). A simple framework for contrastive learning of visual representations. In International conference on machine learning (pp. 1597-1607). PmLR.

SimCLR Pre-Training

Transformations t and t' are both sampled from the same type of transformations $\mathcal{T}$:

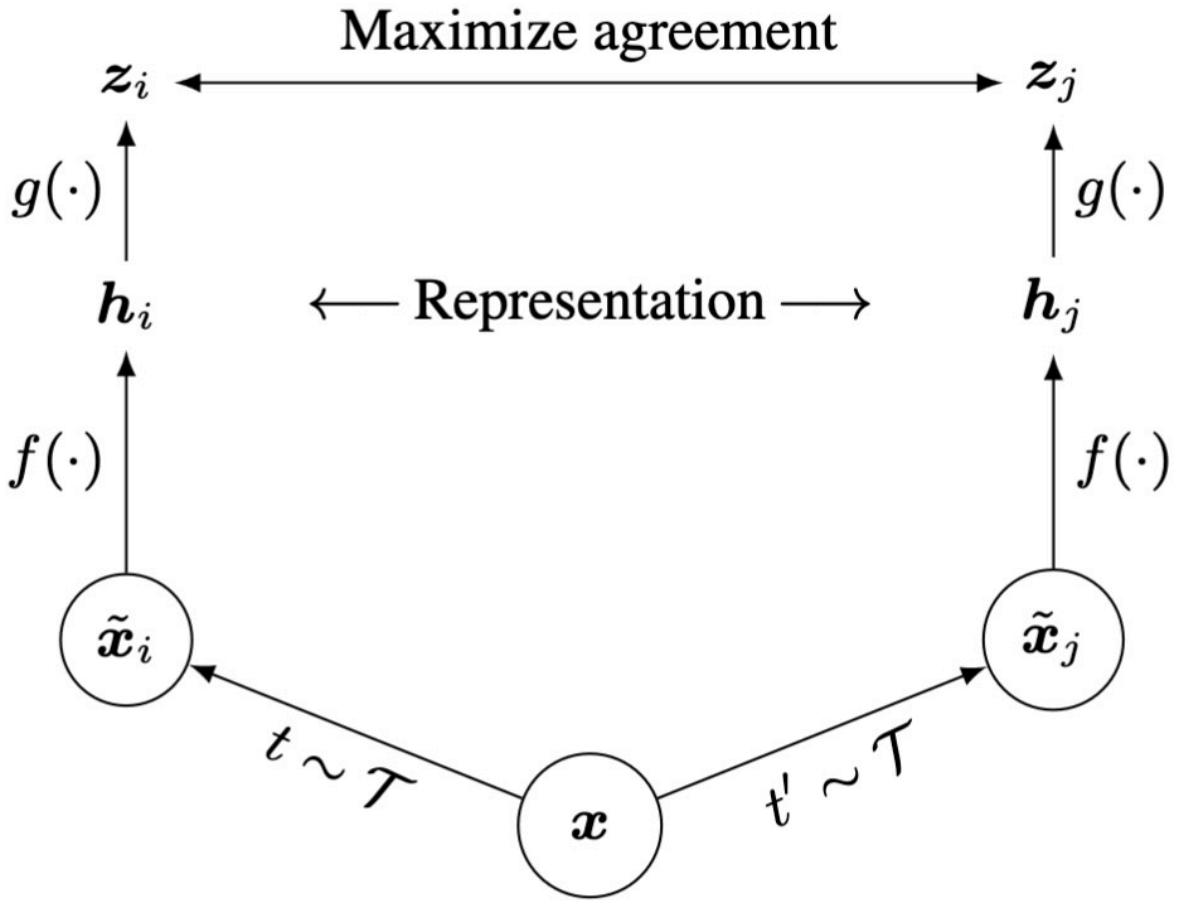


Image source: Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020, November). A simple framework for contrastive learning of visual representations. In International conference on machine learning (pp. 1597-1607). PmLR.

SimCLR: Contrastive Objective

SimCLR (Chen et al., 2020) trains a visual encoder f_θ using a **contrastive loss** that pulls together different augmented views of the same image (positive pairs) and pushes apart views from different images (negative pairs).

For a mini-batch of N images, each augmented twice, the **normalized temperature-scaled cross-entropy (NT-Xent) loss** for a positive pair (i, j) is:

$$\mathcal{L}_{ij} = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

where $\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$ is cosine similarity, τ is a temperature hyperparameter, and $\mathbf{z} = g(f_\theta(\mathbf{x}))$ is the projection of the encoder output through a small MLP head g.

SimCLR with a ViT Backbone

Originally proposed with a ResNet backbone, SimCLR can equally use a **ViT backbone**:

- The ViT encoder f_θ processes augmented patch sequences
- The class token output $\mathbf{z}_{\text{cls}}^{(L)}$ serves as the image representation
- A projection MLP head maps it to the contrastive embedding space

After pre-training, the projection head is discarded and the encoder is fine-tuned on a downstream task with labels.

Contrastive Language-Image Pretraining

Pretraining using semantic similarities between images and text:

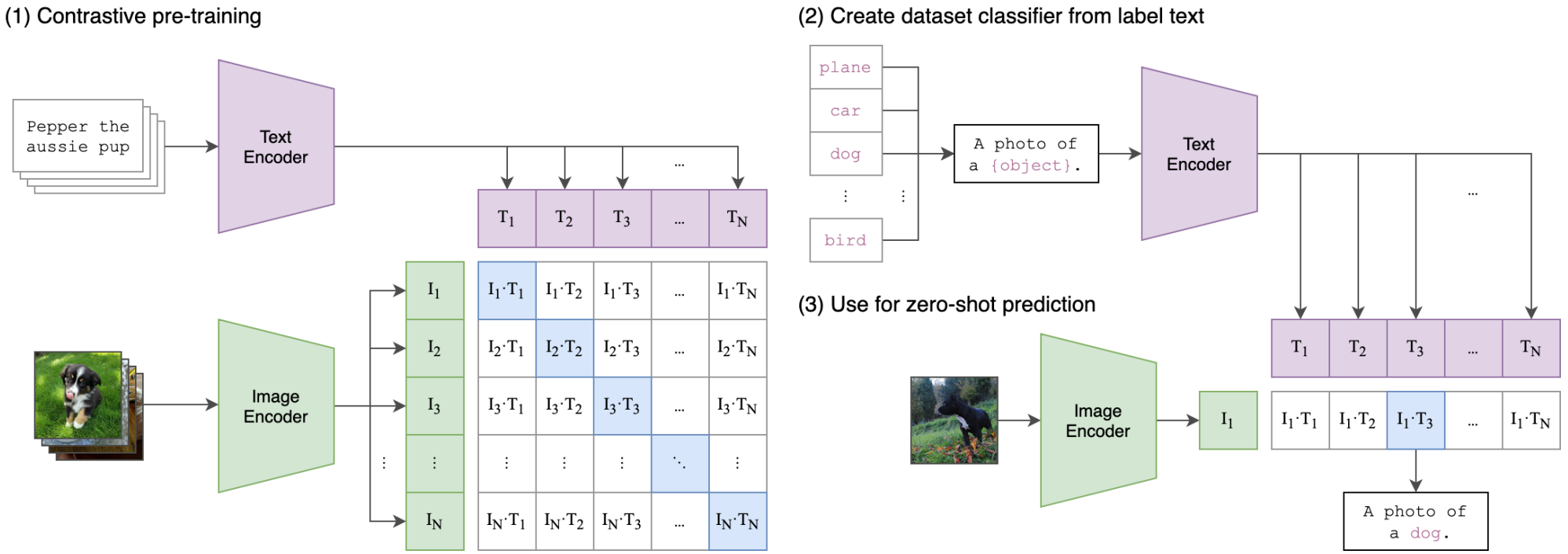


Image source: OpenAI, MIT , via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Contrastive_Language-Image_Pretraining.png

CLIP: Architecture

CLIP (Contrastive Language–Image Pre-training, Radford et al., 2021) trains two separate encoders:

- **Image encoder** f_θ: a ViT (or ResNet) that maps an image to an embedding $\mathbf{v} = f_\theta(\mathbf{x}) \in \mathbb{R}^d$
- **Text encoder** g_ϕ: a Transformer that maps a text caption to an embedding $\mathbf{u} = g_\phi(t) \in \mathbb{R}^d$

Both embeddings are L_2-normalized before computing similarity. The model is trained on image–caption pairs collected from the internet.

CLIP: Training Objective

For a mini-batch of N image–caption pairs $\{(\mathbf{x}_i, t_i)\}_{i=1}^{N}$:

$$\mathbf{v}_i = \frac{f_\theta(\mathbf{x}_i)}{\|f_\theta(\mathbf{x}_i)\|}, \quad \mathbf{u}_i = \frac{g_\phi(t_i)}{\|g_\phi(t_i)\|}$$

The similarity matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$ with $S_{ij} = \mathbf{v}_i^\top \mathbf{u}_j / \tau$ is computed, and the loss maximizes the diagonal entries (correct pairs):

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}[\mathcal{L}_{\text{img}\to\text{txt}} + \mathcal{L}_{\text{txt}\to\text{img}}]$$

where each term is a cross-entropy loss over the N rows or columns of $\mathbf{S}$ with the identity permutation as the ground-truth assignment.

CLIP: The Two Directional Losses

Image → Text loss $\mathcal{L}_{\text{img}\to\text{txt}}$: For each image i, find its matching caption among all N texts in the batch:

$$\mathcal{L}_{\text{img}\to\text{txt}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ij})}$$

"Given image i, which of the N texts belongs to it?" — softmax is computed across the columns of row i.

CLIP: The Two Directional Losses

Text → Image loss $\mathcal{L}_{\text{txt}\to\text{img}}$: For each caption i, find its matching image among all N images in the batch:

$$\mathcal{L}_{\text{txt}\to\text{img}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ji})}$$

"Given text i, which of the N images belongs to it?" — softmax is computed across the rows of column i.

CLIP: The Two Directional Losses

The final loss $\mathcal{L}_{\text{CLIP}} = \frac{1}{2}[\mathcal{L}_{\text{img}\to\text{txt}} + \mathcal{L}_{\text{txt}\to\text{img}}]$ is **symmetric**: both encoders are trained jointly, pushing matching pairs together and pushing all $N^2 - N$ non-matching pairs apart.

CLIP: Zero-Shot Inference

A key capability of CLIP is **zero-shot image classification** — no task-specific fine-tuning is needed:

1. Define K class names: `"a photo of a cat"`, `"a photo of a dog"`, …
2. Encode all K text prompts with the text encoder: $\mathbf{u}_1, \ldots, \mathbf{u}_K$
3. Encode the query image: $\mathbf{v}$
4. Predict the class with the highest cosine similarity:

$$\hat{k} = \arg\max_k \mathbf{v}^\top \mathbf{u}_k$$

This works because both encoders are trained to align images and their natural-language descriptions in the same embedding space.

ViT as a Dense Feature Extractor

Beyond classification, ViT is increasingly used as a **dense feature extractor** for spatial tasks:

How it works:

- After the last encoder layer, the N patch token representations $\mathbf{z}_1^{(L)}, \ldots, \mathbf{z}_N^{(L)}$ are reshaped to a 2D feature map of size $\frac{H}{P} \times \frac{W}{P} \times d$
- This feature map is fed to a **task-specific head** — a decoder, segmentation head, or depth regression network

DINOv2: Motivation

CLIP requires paired image–text data. Can we pre-train a strong visual encoder using **only images** — without any text supervision or labels?

DINO (Caron et al., 2021) introduced self-distillation with no labels for ViT. **DINOv2** (Oquab et al., 2023) scales this idea with:

- A **curated high-quality dataset** (LVD-142M: 142 million images)
- **Improved training recipe**: distillation + masked image modeling combined
- Resulting in **general-purpose visual features** that transfer to dense prediction tasks (depth estimation, segmentation) with frozen backbones and simple task heads or probes

DINOv2: Self-Distillation Principle

DINOv2 uses a **student–teacher framework** without any labels:

- **Student network** f_s: updated by gradient descent
- **Teacher network** f_t: updated by an **exponential moving average (EMA)** of the student weights:

$$\theta_t \leftarrow \lambda\theta_t + (1 - \lambda)\theta_s$$

The teacher is never trained directly — it is a slowly changing copy of the student. This provides **stable training targets** and prevents representational collapse.

The student is trained to predict the teacher's output for different augmented views of the same image — a form of **self-distillation**.

DINOv1: Method and Loss

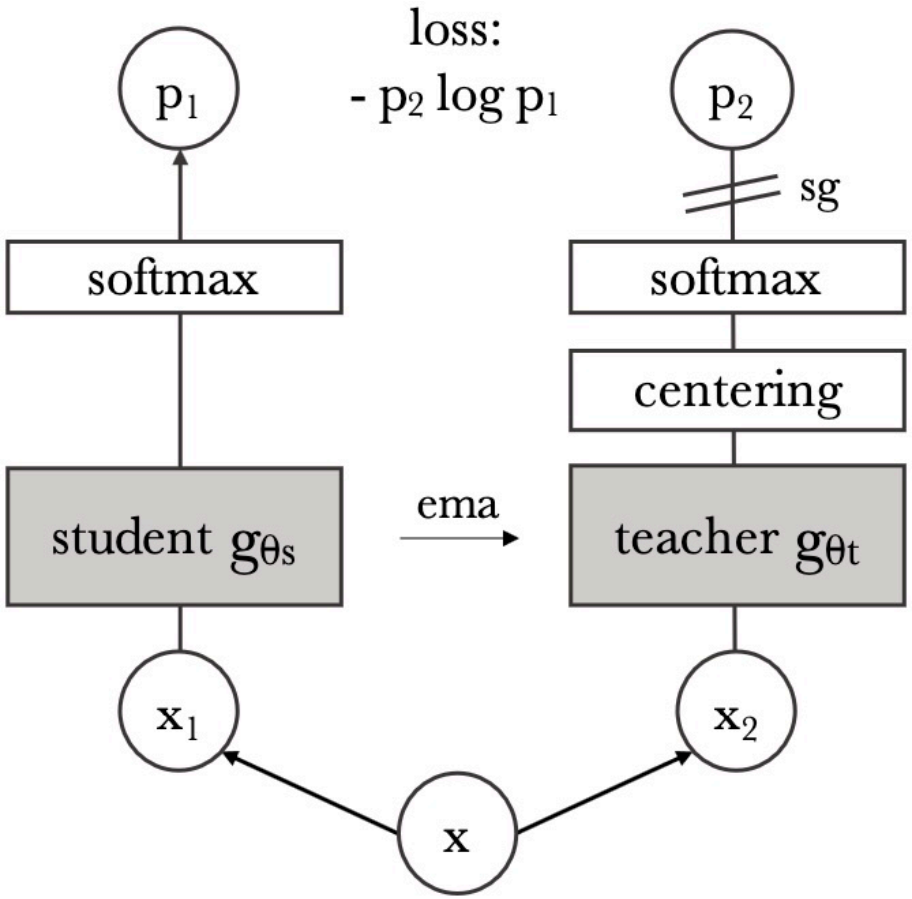


Image source: Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., & Joulin, A. (2021). Emerging properties in self-supervised vision transformers. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 9650-9660).

DINOv2: Training Objective

DINOv2 trains the student to match the teacher's output distributions for different augmented views of the same image. The total loss combines three terms:

DINO loss (image-level, class token): cross-entropy between student and teacher softmax probability vectors $\mathbf{p}_s, \mathbf{p}_t$ over a shared codebook, summed over all crop pairs. Global crops go to both networks; local crops (small views, $\sim 20\%$) only to the student — so the student must predict global semantics from local context:

$$\mathcal{L}_{\text{DINO}} = -\sum_{\text{crop pairs}} \sum_{k} p_{t,k} \log p_{s,k}$$

DINOv2: Training Objective

iBOT loss (patch-level): applies the same cross-entropy distillation to individual masked patch tokens, giving the model spatially structured representations.

KoLeo regularization: encourages diverse embeddings across mini-batch samples to prevent collapse.

Overall loss in DINOv2:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{DINO}} + \lambda_{\text{iBOT}}\,\mathcal{L}_{\text{iBOT}} + \lambda_{\text{KoLeo}}\,\mathcal{L}_{\text{KoLeo}}$$

DINOv2: Curated Dataset LVD-142M

A key contribution of DINOv2 is the **curated training dataset** LVD-142M:

- Collected from publicly available image datasets and the web
- **Deduplication**: near-duplicate images are removed to prevent memorization
- **Curation via retrieval**: images from small curated seed sets (e.g., ImageNet) are used to retrieve semantically similar images from a large uncurated pool — ensuring thematic diversity and quality
- Final dataset: **142 million high-quality images** covering a wide range of visual concepts

This curation process avoids the noise inherent in raw web-scraped data while remaining label-free — the model never sees any class annotations during pre-training.

DINOv2: Curated Dataset LVD-142M

Data processing for collecting the LVD-142M dataset:

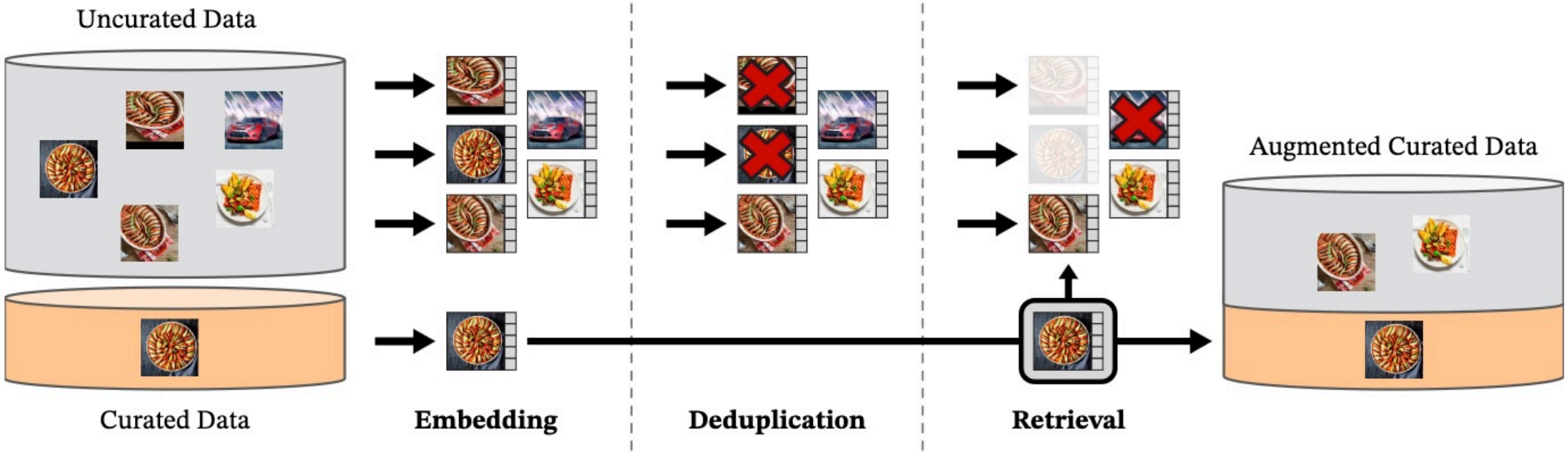


Image source: Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193.

DINOv2: Feature Quality

DINOv2 produces **general-purpose visual features** that can be used as frozen backbone features:

- **Semantic segmentation**: patch token features can be clustered to segment objects without any segmentation supervision — the features are already semantically organized in space
- **Depth estimation**: simple heads on DINOv2 patch features achieve strong monocular depth estimation results
- **Classification**: a linear classifier on top of frozen DINOv2 features achieves strong results across many benchmarks

DINOv2: Feature Quality

First components of PCA on the encoded feature patches:

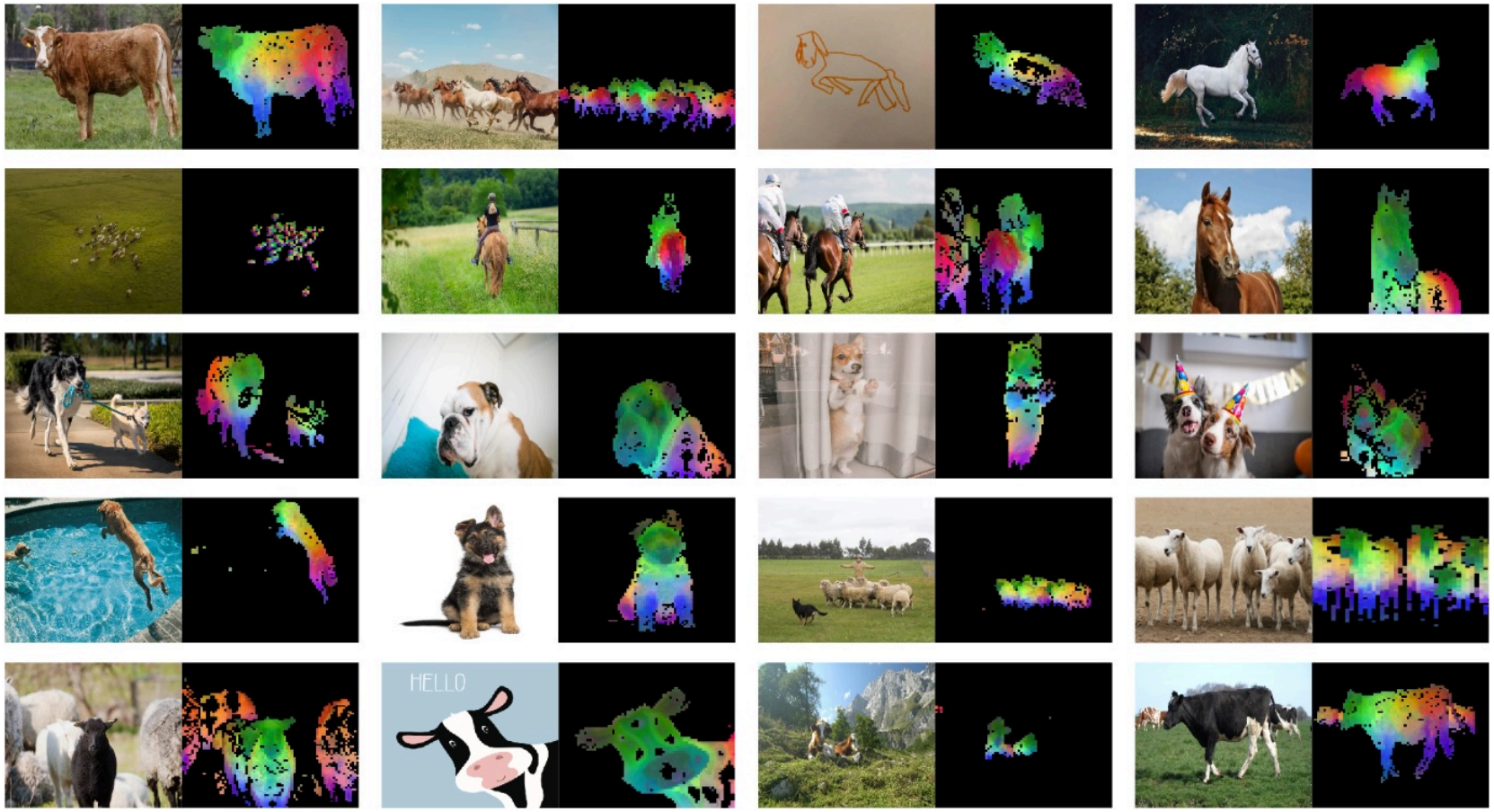


Image source: Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193.

DINOv2: Dense Prediction With Frozen Backbones

A property of DINOv2 is that its patch token representations are **spatially structured**:

- Patches in semantically similar regions produce similar feature vectors
- The 2D layout of patch features resembles a semantic segmentation map — before any task-specific training

This allows dense prediction tasks (segmentation, depth) to be solved with a simple probe or task head applied to the patch token grid while keeping the backbone frozen.

This is in contrast to supervised classification pre-training where the patch tokens are not explicitly trained to be spatially meaningful — only the class token is trained.

DINOv2: Dense Prediction With Frozen Backbones

Examples for semantic segmentation and depth estimation on the feature maps:

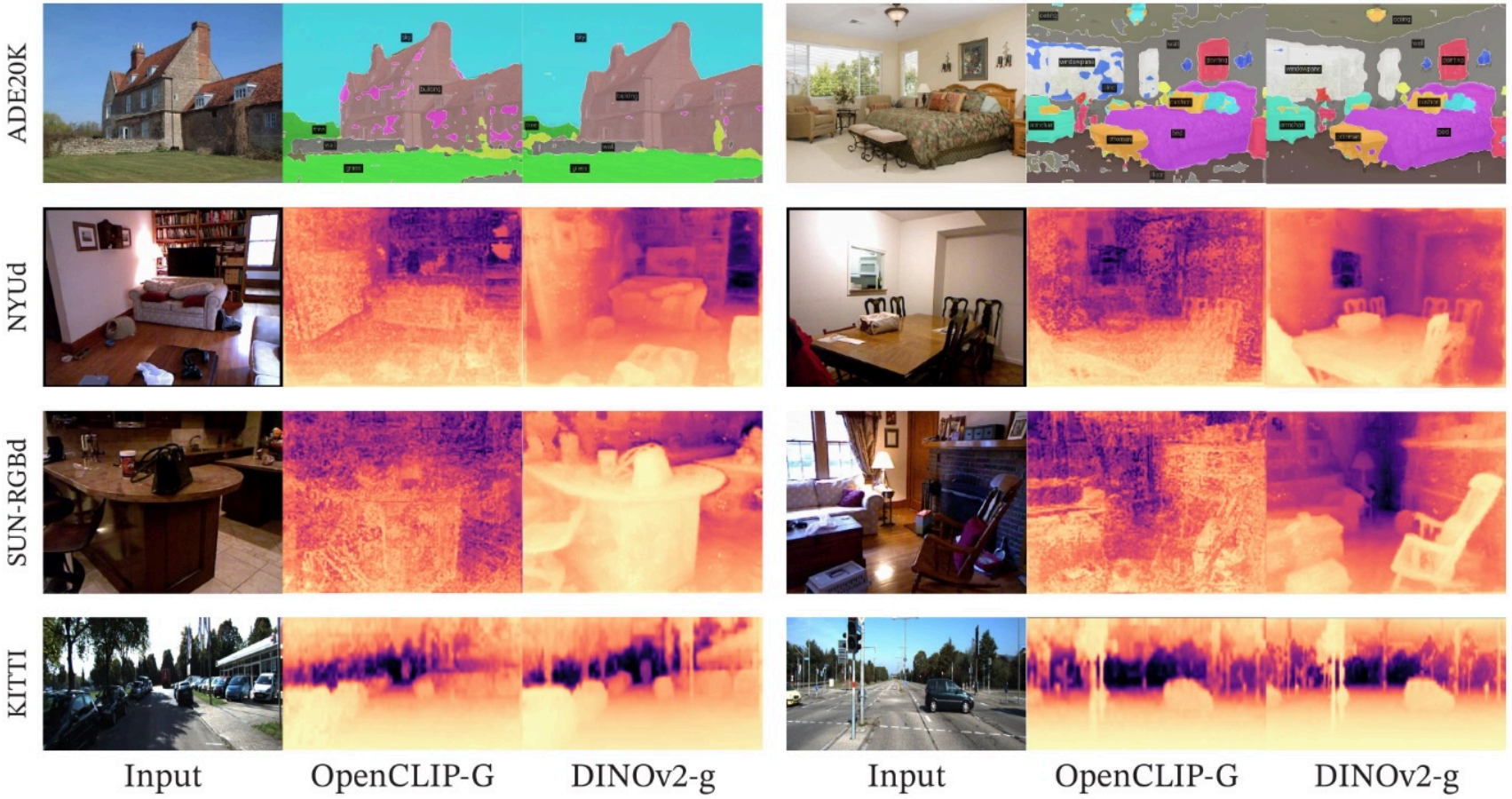


Image source: Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193.

Applications of Pre-Trained ViT Features

A pre-trained ViT can be used for many downstream tasks:

Task	How ViT features are used
Image classification	Linear probe or fine-tune on class token
Object detection	Patch token grid as backbone for region proposals
Semantic segmentation	Linear probe or decoder head on patch token grid
Monocular depth estimation	Linear probe or small decoder on patch tokens
Image retrieval	Cosine similarity in class token embedding space
Few-shot learning	Metric learning in frozen embedding space

Pre-Training Strategies: Summary

Method	Supervisory Signal	Data Required	Key Strength
Supervised	Class labels	Labeled images	Strong classification features
MAE	Pixel reconstruction	Unlabeled images	Efficient, scalable
SimCLR	View consistency	Unlabeled images	No labels, strong representations
CLIP	Image–text alignment	Image–caption pairs	Zero-shot transfer, multimodal
DINOv2	Self-distillation	Curated unlabeled images	Dense spatial features, no labels

Vision Transformer: Chapter Summary

What we covered in this chapter:

1. **From tokens to patches** — images are split into fixed-size patches, linearly projected to patch embeddings, and processed as a sequence
2. **ViT architecture** — patch embedding, class token, 1D learnable positional encoding, stacked encoder blocks, MLP classification head
3. **Self-attention on patches** — every patch attends to every other patch; multi-head attention captures diverse spatial relationships; attention maps are interpretable
4. **CNN vs. ViT** — ViT lacks spatial inductive bias but scales better with data
5. Pretraining strategies: **supervised**, **MAE**, **SimCLR**, **CLIP** and **DINOv2**